Teste 4 da PoC (v5): STEP-GAN Federado com SMPC

Modelo final da PoC: **STEP-GAN** semi-supervisionado treinado com **Aprendizado Federado** entre 3 silos bancários e agregação por **SMPC** (compartilhamento aditivo de segredos em aritmética modular).

O que mudou em relação à versão anterior (comentários da revisão)

1. Várias sementes. O experimento completo roda com `SEMENTES` (5 a 10) e reporta média ± desvio-padrão e IC 95% de todas as métricas, além da tabela por semente.

2. SMPC revisado. A versão anterior usava p = 2^31−1 com escala 2^24, o que limitava a **soma** dos clientes a |Σ| < 64. Acima disso o valor "dava a volta" no módulo **sem nenhum aviso** (ex.: 3 clientes com peso 30 viravam −12,67). Ela também **truncava** em vez de arredondar e, na célula da PaySim, usava ruído float (`randn*100`) em vez de partes modulares. Agora:

- p = 2^61−1 (primo de Mersenne), com faixa |Σ| < 2^36 ≈ 6,9·10^10, e **verificação explícita de estouro** antes de cada compartilhamento;
- codificação em ponto fixo com **arredondamento**, e números negativos representados como p − |x| (decodificados como negativos quando x > p/2);
- todas as somas são reduzidas mod p a cada passo, para nunca estourar o int64;
- máscaras geradas por **CSPRNG** (`os.urandom`). Como as máscaras se cancelam exatamente, o agregado não depende delas e a reprodutibilidade é preservada;
- fluxo das partes explícito: cliente k → parte j para o nó j → cada nó soma só o que recebeu → servidor soma as parciais e vê apenas o agregado;
- **autoteste** no início do script (negativos, valores grandes, estouro e a falha da configuração antiga).

3. STEP-GAN com condicionamento por limiares e vários geradores. São K = `len(DEGRAUS)` geradores (2 a 3). Cada gerador cobre um **degrau** [τ_min, τ_max] da saída do Discriminador e recebe o limiar τ como **condição**: G_k(z, τ), com o alvo D(G_k(z, τ)) ≈ τ. O Discriminador é **graduado**: além de normal real → 0,9 e fraude real → 0,1, ele aprende a atribuir τ às amostras do degrau τ. Assim os geradores produzem pseudo-anomalias em níveis ordenados de "normalidade" (de longe a perto da fronteira), e o Discriminador aprende essa escala em degraus. O gráfico "Condicionamento por limiares" mostra τ pedido × D obtido. Os geradores também passam a ser **agregados via SMPC** (antes só o Discriminador era).

Por que graduado: rotulando todas as amostras geradas como 0,1, o condicionamento só funcionava para τ ≤ ~0,5 (acima disso o jogo adversarial prendia D(G) em ~0,5) ou o desempenho caía (PR-AUC 0,52). Com o Discriminador graduado, o condicionamento fica efetivo (correlação τ×D ≈ 0,97) e o PR-AUC volta a ~0,85.

Também corrigido: split 60/20/20 com scaler ajustado **só no treino** e limiar escolhido **só na validação**, também na PaySim; FL em `NUM_RODADAS` rodadas (mesmo total de 35 épocas), com o modelo global escolhido pela melhor rodada na validação; throughput medido com sincronização da GPU.

```
# -*- coding: utf-8 -*-
"""
Teste 4 da PoC (v5): Detecção de Fraude com STEP-GAN Federado + SMPC
- STEP-GAN: K geradores condicionados por limiares (degraus da saída do Discriminador)
- Aprendizado Federado (3 silos) com agregação SMPC modular (p = 2^61 - 1)
- Avaliação com várias sementes (média ± desvio-padrão)
"""

import os
import copy
import time
import random

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score, confusion_matrix,
                             precision_recall_curve)

try:
    import torch
    import torch.nn as nn
    import torch.optim as optim
    from torch.utils.data import DataLoader, TensorDataset
except ImportError:
```

```
    raise ImportError("PyTorch não está instalado. Execute: pip install torch")

# ======================================================================
# --- 0. CONFIGURAÇÃO ---
# ======================================================================
SEMENTES = [42, 43, 44, 45, 46]    # 5 a 10 sementes (comentário 1)
NUM_CLIENTES = 3
NUM_RODADAS = 5                     # Rodadas de comunicação do FL
EPOCAS_LOCAIS = 7                   # 5 x 7 = 35 épocas, mesmo total da versão anterior

# Hiperparâmetros do Teste 4
HP_BATCH_SIZE = 512
HP_LR_G = 0.0002
HP_LR_D = 0.0001
HP_LATENT_DIM = 100
HP_WEIGHT_DECAY = 1e-4
PESO_FRAUDE = 15.0                  # Loss ponderada para fraudes reais
ALVO_NORMAL = 0.9                   # Label smoothing: normal real
ALVO_ANOMALIA = 0.1                 # Fraude real e amostras geradas
PRECISAO_MINIMA = 0.70              # Limiar: maior recall com precisão >= 70% (na validação)
RAZAO_NORMAIS_POR_FRAUDE = 200      # Amostragem: 200 normais para cada fraude

# STEP-GAN: um gerador por degrau da saída do Discriminador (comentário 3)
# Degraus entre ALVO_ANOMALIA (0.1) e ALVO_NORMAL (0.9): pseudo-anomalias de "longe" a "perto" da fronteira
DEGRAUS = [(0.10, 0.35), (0.35, 0.60), (0.60, 0.85)]
DIM_CONDICAO = 16
# Discriminador graduado: a amostra de G_k(z, τ) recebe alvo (τ - MARGEM_GRADUADA) no Discriminador, em vez de 0.1.
# Assim D e G_k concordam sobre o degrau e o condicionamento passa a ser efetivo. Com todas as amostras geradas
# rotuladas como 0.1 (versão anterior), o condicionamento só funcionava para τ <= ~0.5 ou o desempenho caía.
# Ablação (Credit Card, semente 42): margem 0.0 -> PR-AUC 0.855 e correlação τ×D ~0.97; margem 0.15 -> PR-AUC 0.668.
DISCRIMINADOR_GRADUADO = True
MARGEM_GRADUADA = 0.0

# SMPC (comentário 2)
SMPC_MODULO = 2**61 - 1             # Primo de Mersenne; partes em [0, p) e somas de 2 partes < 2^62 cabem em int64
SMPC_BITS_PRECISAO = 24             # Resolução do ponto fixo: 2^-24 ≈ 6e-8
SMPC_ESCALA = 2 ** SMPC_BITS_PRECISAO
SMPC_LIMITE_SOMA = (SMPC_MODULO // 2) / SMPC_ESCALA   # |Σ| representável ≈ 6.9e10

DATASETS = ['paysim', 'creditcard']
CAMINHOS = {
    'creditcard': [
        '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/creditcard/creditcard.csv',
        'base_de_dados/creditcard.zip',        # Executando de dentro de PoC/
        'PoC/base_de_dados/creditcard.zip',    # Executando da raiz do repositório
    ],
    'paysim': [
        '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/paysim.csv',
    ],
}


def seed_everything(seed):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False


sns.set_style("whitegrid")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"[SETUP] Dispositivo PyTorch: {device}")


def sincronizar():
    """Garante que o cronômetro só pare depois que a GPU terminar (CUDA é assíncrono)."""
    if device.type == 'cuda':
        torch.cuda.synchronize()


try:
    from google.colab import drive
    drive.mount('/content/drive')
except ImportError:
    print("Fora do Google Colab: usando apenas caminhos locais.")
```

```
[SETUP] Dispositivo PyTorch: cuda
Mounted at /content/drive
```

```
# ======================================================================
# --- 1. PROTOCOLO SMPC: COMPARTILHAMENTO ADITIVO EM Z_p (comentário 2) ---
# ======================================================================

def codificar_ponto_fixo(v):
    """
    float64 -> Z_p. Arredonda para o inteiro mais próximo de v * 2^24 e representa negativos como p - |x|.
    Ex.: -1.5 -> round(-1.5 * 2^24) = -25165824 -> p - 25165824.
    """
    return torch.remainder(torch.round(v * SMPC_ESCALA).to(torch.int64), SMPC_MODULO)


def decodificar_ponto_fixo(x):
    """Z_p -> float64. Valores acima de p/2 são interpretados como negativos."""
    x = torch.where(x > SMPC_MODULO // 2, x - SMPC_MODULO, x)
    return x.to(torch.float64) / SMPC_ESCALA


def mascaras_csprng(forma):
    """Inteiros uniformes em [0, p) gerados com os.urandom (CSPRNG do sistema operacional)."""
    n = int(np.prod(forma))
    brutos = np.frombuffer(os.urandom(8 * n), dtype=np.uint64) & np.uint64(SMPC_MODULO)  # 61 bits
    brutos = brutos % np.uint64(SMPC_MODULO)                                             # mapeia p -> 0
    return torch.from_numpy(brutos.astype(np.int64)).view(forma)


def soma_modular(tensores):
    """Soma reduzindo mod p a cada passo: cada parcela < 2^61, então a soma de duas < 2^62 nunca estoura o int64."""
    total = torch.zeros_like(tensores[0])
    for t in tensores:
        total = torch.remainder(total + t, SMPC_MODULO)
    return total


def gerar_partes(x_int, n):
    """Divide x (já em Z_p) em n partes aleatórias com Σ partes ≡ x (mod p). Cada parte isolada é uniforme em Z_p."""
    mascaras = [mascaras_csprng(x_int.shape) for _ in range(n - 1)]
    ultima = x_int
    for m in mascaras:
        ultima = torch.remainder(ultima - m, SMPC_MODULO)  # diferença em (-p, p): sem estouro
    return mascaras + [ultima]


def verificar_faixa(v, n):
    """
    Checagem de estouro feita por cada cliente antes de compartilhar: se todo |v_k| <= LIMITE / n,
    então |Σ v_k| < p / 2 e a decodificação de negativos continua correta.
    """
    maximo = v.abs().max().item()
    if maximo > SMPC_LIMITE_SOMA / n:
        raise ValueError(f"Estouro do módulo SMPC: |valor| = {maximo:.3e} > {SMPC_LIMITE_SOMA / n:.3e}. "
                         f"Aumente SMPC_MODULO ou reduza SMPC_BITS_PRECISAO.")


def achatar(estado):
    """Concatena todos os tensores de ponto flutuante do estado (pesos e buffers do BatchNorm) em float64."""
    return torch.cat([t.detach().cpu().to(torch.float64).flatten()
                      for t in estado.values() if t.is_floating_point()])


def desachatar(vetor, estado_ref, estados_clientes):
    novo, pos = {}, 0
    for chave, t in estado_ref.items():
        if t.is_floating_point():
            novo[chave] = vetor[pos:pos + t.numel()].view(t.shape).to(t.dtype)
            pos += t.numel()
        else:
            # num_batches_tracked do BatchNorm: contador inteiro sem informação dos dados (não é usado com momentum)
            novo[chave] = torch.stack([e[chave] for e in estados_clientes]).max()
    return novo


def agregar_smpc(estados_clientes, pesos):
    """
    FedAvg ponderado via SMPC com n nós (os próprios clientes):
      1. Cliente k: pondera (w_k * n_k/N), verifica a faixa, codifica em Z_p e gera n partes; a parte j vai para o nó j.
      2. Nó j: soma (mod p) apenas as partes que recebeu, que são individualmente uniformes (não revelam nada).
      3. Servidor: soma (mod p) as n somas parciais e decodifica. Ele só conhece o agregado Σ_k w_k n_k/N.
    """
    n = len(estados_clientes)
    t0 = time.perf_counter()
    caixas = [[] for _ in range(n)]                     # caixas[j]: partes recebidas pelo nó j
```

```
    for estado, peso in zip(estados_clientes, pesos):
        v = achatar(estado) * peso
        verificar_faixa(v, n)
        for j, parte in enumerate(gerar_partes(codificar_ponto_fixo(v), n)):
            caixas[j].append(parte)
    somas_parciais = [soma_modular(caixas[j]) for j in range(n)]
    agregado = decodificar_ponto_fixo(soma_modular(somas_parciais))
    t_smpc = time.perf_counter() - t0

    referencia = sum(achatar(e) * p for e, p in zip(estados_clientes, pesos))   # FedAvg em claro
    erro_max = (agregado - referencia).abs().max().item()
    bytes_cliente = n * agregado.numel() * 8           # (n-1) partes aos outros nós + 1 soma parcial
    return desachatar(agregado, estados_clientes[0], estados_clientes), dict(
        t_smpc=t_smpc, erro_max=erro_max, bytes_cliente=bytes_cliente)


def autoteste_smpc():
    print("--- Autoteste do SMPC ---")
    n, pesos = 3, [1 / 3] * 3
    valores = [torch.tensor([-1.5, 30.0, -30.0, 1595.5, -2.0e7, 7e-8, 0.123456789], dtype=torch.float64) * f
               for f in (1.0, 0.5, -0.25)]
    esperado = sum(v * p for v, p in zip(valores, pesos))
    caixas = [[] for _ in range(n)]
    for v, p in zip(valores, pesos):
        for j, parte in enumerate(gerar_partes(codificar_ponto_fixo(v * p), n)):
            caixas[j].append(parte)
    obtido = decodificar_ponto_fixo(soma_modular([soma_modular(c) for c in caixas]))
    erro = (obtido - esperado).abs().max().item()
    tolerancia = n * 0.5 / SMPC_ESCALA
    assert erro <= tolerancia, f"Erro de reconstrução {erro} > {tolerancia}"
    print(f"  Reconstrução com negativos e valores grandes: erro máx. {erro:.2e} (tolerância {tolerancia:.2e}) OK")

    parte_isolada = decodificar_ponto_fixo(caixas[0][0])
    print(f"  Uma parte isolada decodificada parece ruído: {parte_isolada[:3].tolist()}")

    try:
        verificar_faixa(torch.tensor([SMPC_LIMITE_SOMA], dtype=torch.float64), n)
        raise AssertionError("A verificação de estouro deveria ter falhado")
    except ValueError:
        print(f"  Estouro detectado antes de compartilhar (limite por cliente {SMPC_LIMITE_SOMA / n:.2e}) OK")

    p_antigo, escala_antiga = 2147483647, 2 ** 24
    soma_antiga = 3 * round(30.0 * escala_antiga) % p_antigo
    soma_antiga = soma_antiga - p_antigo if soma_antiga > p_antigo // 2 else soma_antiga
    print(f"  Configuração antiga (p = 2^31-1): 3 clientes com peso 30 -> média {soma_antiga / escala_antiga / 3:+.2f} "
          f"(esperado +30.00): estouro silencioso corrigido.")


autoteste_smpc()
```

```
--- Autoteste do SMPC ---
  Reconstrução com negativos e valores grandes: erro máx. 3.02e-08 (tolerância 8.94e-08) OK
  Uma parte isolada decodificada parece ruído: [-49954312165.74475, -27793768349.788036, -31028970219.26253]
  Estouro detectado antes de compartilhar (limite por cliente 2.29e+10) OK
  Configuração antiga (p = 2^31-1): 3 clientes com peso 30 -> média -12.67 (esperado +30.00): estouro silencioso corrigido.
```

```
# ======================================================================
# --- 2. DADOS (engenharia de features do Teste 4, sem data leakage) ---
# ======================================================================

def ler_base(nome):
    for caminho in CAMINHOS[nome]:
        if os.path.exists(caminho):
            print(f"Carregando '{caminho}'...")
            return pd.read_csv(caminho)
    return None


def amostrar(df, alvo):
    """Mantém todas as fraudes e RAZAO_NORMAIS_POR_FRAUDE vezes mais transações normais."""
    fraudes = df[df[alvo] == 1]
    normais = df[df[alvo] == 0]
    normais = normais.sample(n=min(len(fraudes) * RAZAO_NORMAIS_POR_FRAUDE, len(normais)), random_state=42)
    return pd.concat([normais, fraudes]).sample(frac=1, random_state=42).reset_index(drop=True)


def preparar_creditcard(df):
    df = amostrar(df.dropna(subset=['Class']), 'Class')
    df = df.drop(columns=['Time'], errors='ignore')      # Contador monotônico: ruído para o GAN
    df['amount_log'] = np.log1p(df['Amount'].clip(lower=0))
    return df.drop(columns=['Amount']), 'Class'
```

```
def preparar_paysim(df):
    df = amostrar(df.dropna(subset=['isFraud']), 'isFraud')
    df['orig_balance_zero'] = (df['oldbalanceOrg'] == 0).astype(int)
    df['dest_balance_zero'] = (df['oldbalanceDest'] == 0).astype(int)
    for col in ['amount', 'oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest']:
        df[col + '_log'] = np.log1p(df[col].clip(lower=0))
    df['errorBalanceOrig'] = df['newbalanceOrig'] + df['amount'] - df['oldbalanceOrg']
    df['errorBalanceDest'] = df['oldbalanceDest'] + df['amount'] - df['newbalanceDest']
    df = df.drop(columns=['amount', 'oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest'])
    df = df.drop(columns=[c for c in ['nameOrig', 'nameDest', 'isFlaggedFraud'] if c in df.columns])
    return pd.get_dummies(df, columns=['type'], drop_first=True, dtype=float), 'isFraud'


PREPARADORES = {'creditcard': preparar_creditcard, 'paysim': preparar_paysim}


def carregar_dados(nome):
    df = ler_base(nome)
    if df is None:
        return None
    df, alvo = PREPARADORES[nome](df)
    X = df.drop(columns=[alvo]).values.astype(np.float32)
    y = df[alvo].values.astype(np.int64)

    # Split 60/20/20 estratificado ANTES de escalar
    X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
    X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=42, stratify=y_temp)

    # MinMaxScaler(-1, 1) alinhado ao Tanh dos geradores, ajustado SÓ no treino
    scaler = MinMaxScaler(feature_range=(-1, 1))
    X_train = scaler.fit_transform(X_train).astype(np.float32)
    X_val = scaler.transform(X_val).astype(np.float32)
    X_test = scaler.transform(X_test).astype(np.float32)

    # Silos bancários: fatias do treino
    fatias = np.array_split(np.arange(len(X_train)), NUM_CLIENTES)
    clientes = [(X_train[i], y_train[i]) for i in fatias]
    pesos_fedavg = [len(i) / len(X_train) for i in fatias]

    print(f"[DADOS] Features: {X_train.shape[1]} | Treino: {len(X_train)} (fraudes: {int(y_train.sum())}) "
          f"| Validação: {len(X_val)} (fraudes: {int(y_val.sum())}) | Teste: {len(X_test)} (fraudes: {int(y_test.sum())})")
    return dict(nome=nome, input_dim=X_train.shape[1], clientes=clientes, pesos_fedavg=pesos_fedavg,
                X_val=X_val, y_val=y_val, X_test=X_test, y_test=y_test)
```

```
# ====================================================================
# --- 3. STEP-GAN: GERADORES CONDICIONADOS POR LIMIARES (comentário 3) ---
# ====================================================================

class GeradorCondicional(nn.Module):
    """G_k(z, τ): gera pseudo-anomalias cuja saída no Discriminador deve ficar em τ (limiar do degrau k)."""

    def __init__(self, input_dim, latent_dim=HP_LATENT_DIM):
        super().__init__()
        self.condicao = nn.Sequential(nn.Linear(1, DIM_CONDICAO), nn.LeakyReLU(0.2))
        self.net = nn.Sequential(
            nn.Linear(latent_dim + DIM_CONDICAO, 128), nn.BatchNorm1d(128), nn.LeakyReLU(0.2),
            nn.Linear(128, 256), nn.BatchNorm1d(256), nn.LeakyReLU(0.2),
            nn.Linear(256, 512), nn.BatchNorm1d(512), nn.LeakyReLU(0.2),
            nn.Linear(512, input_dim), nn.Tanh()
        )

    def forward(self, z, tau):
        return self.net(torch.cat([z, self.condicao(tau)], dim=1))


class Discriminator(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 512), nn.LeakyReLU(0.2), nn.Dropout(0.3),
            nn.Linear(512, 256), nn.LeakyReLU(0.2), nn.Dropout(0.3),
            nn.Linear(256, 64), nn.LeakyReLU(0.2), nn.Dropout(0.2),
            nn.Linear(64, 1), nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x)


def amostrar_tau(k, n):
    """Limiar τ ~ U(degrau k): a condição do gerador k."""
```

```
    minimo, maximo = DEGRAUS[k]
    return torch.rand(n, 1, device=device) * (maximo - minimo) + minimo


def custom_bce_loss(pred, target, real_labels_flag):
    """BCE com peso condicional: erro em fraude real pesa PESO_FRAUDE vezes mais."""
    loss = nn.BCELoss(reduction='none')(pred, target)
    weights = torch.ones_like(target)
    weights[(target < 0.5) & real_labels_flag] = PESO_FRAUDE
    return (loss * weights).mean()


def novo_estado_global(input_dim):
    return {'D': copy.deepcopy(Discriminator(input_dim).state_dict()),
            'G': [copy.deepcopy(GeradorCondicional(input_dim).state_dict()) for _ in DEGRAUS]}


def treinar_local(estado_global, dados_X, dados_y, input_dim, semente):
    """Treino local do STEP-GAN a partir do modelo global (D + K geradores)."""
    torch.manual_seed(semente)
    K = len(DEGRAUS)
    D = Discriminator(input_dim).to(device)
    D.load_state_dict(estado_global['D'])
    Gs = []
    for estado_G in estado_global['G']:
        G = GeradorCondicional(input_dim).to(device)
        G.load_state_dict(estado_G)
        Gs.append(G)
    D.train()
    for G in Gs:
        G.train()

    opt_D = optim.Adam(D.parameters(), lr=HP_LR_D, betas=(0.5, 0.999), weight_decay=HP_WEIGHT_DECAY)
    opts_G = [optim.Adam(G.parameters(), lr=HP_LR_G, betas=(0.5, 0.999), weight_decay=HP_WEIGHT_DECAY) for G in Gs]
    loader = DataLoader(TensorDataset(torch.from_numpy(dados_X), torch.from_numpy(dados_y).float()),
                        batch_size=HP_BATCH_SIZE, shuffle=True, generator=torch.Generator().manual_seed(semente))

    for _ in range(EPOCAS_LOCAIS):
        for i, (real_data, real_y) in enumerate(loader):
            real_data, real_y = real_data.to(device), real_y.to(device)
            batch_s = real_data.size(0)
            n_k = max(batch_s // K, 2)                  # BatchNorm exige pelo menos 2 amostras
            if batch_s < 2:
                continue

            real_labels = torch.where(real_y.unsqueeze(1) == 0,
                                      torch.tensor(ALVO_NORMAL, device=device),
                                      torch.tensor(ALVO_ANOMALIA, device=device))
            taus = [amostrar_tau(k, n_k) for k in range(K)]
            fakes = [G(torch.randn(n_k, HP_LATENT_DIM, device=device), tau) for G, tau in zip(Gs, taus)]

            # --- Discriminador graduado: normal real -> 0.9; fraude real -> 0.1; amostra do degrau τ -> τ ---
            opt_D.zero_grad()
            loss_real = custom_bce_loss(D(real_data), real_labels, torch.ones_like(real_labels, dtype=torch.bool))
            fake_cat = torch.cat([f.detach() for f in fakes])
            if DISCRIMINADOR_GRADUADO:
                fake_labels = torch.cat(taus).sub(MARGEM_GRADUADA).clamp(min=ALVO_ANOMALIA)
            else:
                fake_labels = torch.full((fake_cat.size(0), 1), ALVO_ANOMALIA, device=device)
            loss_fake = custom_bce_loss(D(fake_cat), fake_labels, torch.zeros_like(fake_labels, dtype=torch.bool))
            (loss_real + loss_fake).backward()
            torch.nn.utils.clip_grad_norm_(D.parameters(), max_norm=1.0)
            opt_D.step()

            # --- Geradores (a cada 2 passos do D): condicionamento D(G_k(z, τ)) -> τ ---
            if i % 2 == 0:
                for G, opt_G, fake, tau in zip(Gs, opts_G, fakes, taus):
                    opt_G.zero_grad()
                    loss_G = nn.BCELoss()(D(fake), tau)
                    loss_G.backward()
                    torch.nn.utils.clip_grad_norm_(G.parameters(), max_norm=1.0)
                    opt_G.step()

    return {'D': {k: v.detach().cpu().clone() for k, v in D.state_dict().items()},
            'G': [{k: v.detach().cpu().clone() for k, v in G.state_dict().items()} for G in Gs]}
```

```
# ======================================================================
# --- 4. APRENDIZADO FEDERADO + AVALIAÇÃO ---
# ======================================================================

def scores_anomalia(estado_D, input_dim, X, tamanho_lote=65536):
    D = Discriminator(input_dim).to(device)
```

```
        D.load_state_dict(estado_D)
        D.eval()
        saida = []
        with torch.no_grad():
            for i in range(0, len(X), tamanho_lote):
                saida.append(D(torch.from_numpy(X[i:i + tamanho_lote]).to(device)).cpu().numpy().ravel())
        return 1.0 - np.concatenate(saida)        # D ≈ 0.9 normal, ≈ 0.1 anomalia


    def escolher_limiar(y_val, s_val):
        """Maior recall com precisão >= PRECISAO_MINIMA na VALIDAÇÃO; se inalcançável, máximo F1."""
        precisions, recalls, thresholds = precision_recall_curve(y_val, s_val)
        validos = np.where(precisions[:-1] >= PRECISAO_MINIMA)[0]
        if len(validos) > 0:
            return thresholds[validos[np.argmax(recalls[validos])]]
        f1 = np.divide(2 * precisions * recalls, precisions + recalls,
                       out=np.zeros_like(precisions), where=(precisions + recalls) != 0)
        return thresholds[np.argmax(f1[:-1])]


    def executar_federado(dados, semente):
        """FL com NUM_RODADAS rodadas. D e os K geradores são agregados via SMPC. Retorna o global da melhor rodada."""
        input_dim = dados['input_dim']
        seed_everything(semente)
        estado_global = novo_estado_global(input_dim)
        historico, melhor = [], None

        for rodada in range(1, NUM_RODADAS + 1):
            sincronizar()
            t0 = time.perf_counter()
            locais = [treinar_local(estado_global, X_k, y_k, input_dim, semente * 1000 + rodada * 10 + k)
                      for k, (X_k, y_k) in enumerate(dados['clientes'])]
            sincronizar()
            t_treino = time.perf_counter() - t0

            # Um pacote por cliente com D e todos os geradores: uma única agregação SMPC por rodada
            pacotes = [{**{f'D.{c}': t for c, t in l['D'].items()},
                        **{f'G{k}.{c}': t for k, g in enumerate(l['G']) for c, t in g.items()}} for l in locais]
            agregado, custo = agregar_smpc(pacotes, dados['pesos_fedavg'])
            estado_global = {'D': {c[2:]: t for c, t in agregado.items() if c.startswith('D.')},
                             'G': [{c[len(f'G{k}.'):]: t for c, t in agregado.items() if c.startswith(f'G{k}.')}
                                   for k in range(len(DEGRAUS))]}

            pr_auc_val = average_precision_score(dados['y_val'], scores_anomalia(estado_global['D'], input_dim, dados['X_val']))
            historico.append(dict(rodada=rodada, pr_auc_val=pr_auc_val, t_treino=t_treino, **custo))
            print(f"  Rodada {rodada}/{NUM_RODADAS} | PR-AUC val {pr_auc_val:.4f} | treino {t_treino:.1f}s "
                  f"| SMPC {1000 * custo['t_smpc']:.0f} ms (erro máx. vs claro {custo['erro_max']:.1e})")
            if melhor is None or pr_auc_val > melhor[0]:
                melhor = (pr_auc_val, rodada, copy.deepcopy(estado_global))

        return melhor[2], melhor[1], pd.DataFrame(historico)


    def avaliar(estado_D, dados):
        input_dim = dados['input_dim']
        limiar = escolher_limiar(dados['y_val'], scores_anomalia(estado_D, input_dim, dados['X_val']))
        sincronizar()
        t0 = time.perf_counter()
        s = scores_anomalia(estado_D, input_dim, dados['X_test'])
        sincronizar()
        t_inf = time.perf_counter() - t0
        y, y_pred = dados['y_test'], (s >= limiar).astype(int)
        return dict(scores=s, y_pred=y_pred, limiar=limiar, **{
            'ROC-AUC': roc_auc_score(y, s), 'PR-AUC': average_precision_score(y, s),
            'Precisão': precision_score(y, y_pred, zero_division=0), 'Recall': recall_score(y, y_pred, zero_division=0),
            'F1': f1_score(y, y_pred, zero_division=0), 'Acurácia': accuracy_score(y, y_pred),
            'TPS': len(y) / t_inf if t_inf > 0 else float('nan')})


    def aderencia_condicionamento(estado_global, input_dim, n=2000):
        """Mede se cada gerador respeita seu limiar: compara τ pedido com D(G_k(z, τ)) obtido."""
        D = Discriminator(input_dim).to(device)
        D.load_state_dict(estado_global['D'])
        D.eval()
        resultado = []
        with torch.no_grad():
            for k, estado_G in enumerate(estado_global['G']):
                G = GeradorCondicional(input_dim).to(device)
                G.load_state_dict(estado_G)
                G.eval()
                tau = amostrar_tau(k, n)
                saida = D(G(torch.randn(n, HP_LATENT_DIM, device=device), tau))
```

```
            resultado.append(pd.DataFrame({'gerador': f'G{k + 1} {DEGRAUS[k]}',
                                           'tau': tau.cpu().numpy().ravel(), 'D': saida.cpu().numpy().ravel()}))
    return pd.concat(resultado, ignore_index=True)
```

```
# ======================================================================
# --- 5. EXPERIMENTO COM VÁRIAS SEMENTES (comentário 1) ---
# ======================================================================

METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
resultados_gerais = {}

for nome_base in DATASETS:
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()} | {len(SEMENTES)} sementes | {NUM_RODADAS} rodadas x {EPOCAS_LOCAIS} épocas "
          f"| {len(DEGRAUS)} geradores")
    print("=" * 78)
    dados = carregar_dados(nome_base)
    if dados is None:
        print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_base]}. Pulando.")
        continue

    linhas, historicos, exemplo = [], [], None
    for semente in SEMENTES:
        print(f"\n--- Semente {semente} ---")
        estado_final, rodada_escolhida, historico = executar_federado(dados, semente)
        av = avaliar(estado_final['D'], dados)
        historico['semente'] = semente
        historicos.append(historico)
        linhas.append({'Semente': semente, 'Rodada escolhida': rodada_escolhida, **{m: av[m] for m in METRICAS},
                       'TPS': av['TPS'], 'SMPC (ms/rodada)': 1000 * historico['t_smpc'].mean(),
                       'Erro máx. SMPC': historico['erro_max'].max()})
        print(f"  -> Rodada escolhida: {rodada_escolhida} | ROC-AUC {av['ROC-AUC']:.4f} | PR-AUC {av['PR-AUC']:.4f} "
              f"| F1 {av['F1']:.4f}")
        if exemplo is None:
            exemplo = dict(semente=semente, av=av,
                           aderencia=aderencia_condicionamento(estado_final, dados['input_dim']))

    df = pd.DataFrame(linhas).set_index('Semente')
    df_hist = pd.concat(historicos, ignore_index=True)

    print("\n>> RESULTADOS POR SEMENTE (teste; limiar escolhido na validação):")
    print(df.to_string(formatters={'Erro máx. SMPC': lambda x: f"{x:.1e}", 'TPS': lambda x: f"{x:,.0f}"},
                       float_format=lambda x: f"{x:.4f}"))

    print(f"\n>> MÉDIA ± DESVIO-PADRÃO ({len(SEMENTES)} sementes) e IC 95% (t de Student):")
    resumo = []
    for m in METRICAS:
        v = df[m].values
        dp = v.std(ddof=1) if len(v) > 1 else 0.0
        meia = stats.t.ppf(0.975, len(v) - 1) * dp / np.sqrt(len(v)) if len(v) > 1 else 0.0
        resumo.append({'Métrica': m, 'Média': v.mean(), 'Desvio-padrão': dp,
                       'IC95% inf': v.mean() - meia, 'IC95% sup': v.mean() + meia})
        print(f"  {m:9s}: {v.mean():.4f} ± {dp:.4f}   IC95% [{v.mean() - meia:.4f}, {v.mean() + meia:.4f}]")
    df_resumo = pd.DataFrame(resumo).set_index('Métrica')

    aderencia = exemplo['aderencia']
    print(f"\n>> CONDICIONAMENTO POR LIMIARES (semente {exemplo['semente']}): τ pedido × D(G_k(z, τ)) obtido")
    for g, grupo in aderencia.groupby('gerador'):
        corr = np.corrcoef(grupo['tau'], grupo['D'])[0, 1] if grupo['D'].std() > 0 else float('nan')
        print(f"  {g}: D médio {grupo['D'].mean():.3f} | erro absoluto médio {np.abs(grupo['D'] - grupo['tau']).mean():.3f}"
              f"| correlação τ×D {corr:+.3f}")

    print(f"\n>> SMPC: {df['SMPC (ms/rodada)'].mean():.0f} ms por rodada | erro máx. vs FedAvg em claro "
          f"{df['Erro máx. SMPC'].max():.1e} | {df_hist['bytes_cliente'].iloc[0] / 1e6:.1f} MB enviados por cliente por roda

    arquivo = f'resultados_teste4_{nome_base}.csv'
    df.to_csv(arquivo)
    print(f"Resultados por semente salvos em '{arquivo}'.")
    resultados_gerais[nome_base] = dict(por_semente=df, resumo=df_resumo, historico=df_hist, aderencia=aderencia)

    # ----- Visualizações -----
    fig, axes = plt.subplots(2, 3, figsize=(22, 11))
    fig.suptitle(f"Teste 4 (v5): STEP-GAN Federado + SMPC ({nome_base}, {len(SEMENTES)} sementes)",
                 fontsize=14, fontweight='bold')

    # 1. Métricas: média ± desvio, com os pontos de cada semente
    ax = axes[0, 0]
    ax.bar(METRICAS, df_resumo['Média'], yerr=df_resumo['Desvio-padrão'], capsize=5, color='#2ca02c', alpha=0.7)
    for i, m in enumerate(METRICAS):
        ax.scatter(np.full(len(df), i) + np.linspace(-0.15, 0.15, len(df)), df[m], color='black', s=15, zorder=3)
    ax.set_ylim(0, 1.05)
    ax.set_title('Métricas no teste (média ± desvio; pontos = sementes)', fontweight='bold')
```

```
    # 2. Convergência do FL
    ax = axes[0, 1]
    for semente, grupo in df_hist.groupby('semente'):
        ax.plot(grupo['rodada'], grupo['pr_auc_val'], color='#1f77b4', alpha=0.3)
    media = df_hist.groupby('rodada')['pr_auc_val'].mean()
    ax.plot(media.index, media.values, color='#1f77b4', lw=2.5, marker='o', label='Média entre sementes')
    ax.set_title('PR-AUC na validação por rodada federada', fontweight='bold')
    ax.set_xlabel('Rodada')
    ax.legend()

    # 3. Condicionamento por limiares
    ax = axes[0, 2]
    cores_g = ['#d62728', '#ff7f0e', '#9467bd']
    for (g, grupo), cor in zip(aderencia.groupby('gerador'), cores_g):
        ax.scatter(grupo['tau'], grupo['D'], s=4, alpha=0.3, color=cor, label=g)
    ax.plot([0, 1], [0, 1], ':', color='gray', label='Ideal: D = τ')
    ax.axhline(ALVO_NORMAL, color='#2ca02c', ls='--', lw=1, label='Alvo normal (0.9)')
    ax.set_xlabel('Limiar pedido τ')
    ax.set_ylabel('Saída do Discriminador D(G_k(z, τ))')
    ax.set_title(f'Condicionamento por limiares (semente {exemplo["semente"]})', fontweight='bold')
    ax.legend(fontsize=8, markerscale=3)

    # 4. Distribuição dos escores
    ax = axes[1, 0]
    s, y = exemplo['av']['scores'], dados['y_test']
    sns.histplot(s[y == 0], color='#1f77b4', bins=50, alpha=0.6, label='Legítima', ax=ax, stat='density')
    sns.histplot(s[y == 1], color='#d62728', bins=50, alpha=0.6, label='Fraude', ax=ax, stat='density')
    ax.axvline(exemplo['av']['limiar'], color='black', ls='--', lw=2, label=f"Limiar: {exemplo['av']['limiar']:.3f}")
    ax.set_yscale('log')
    ax.set_title(f'Escore de anomalia no teste (semente {exemplo["semente"]})', fontweight='bold')
    ax.legend()

    # 5. Matriz de confusão
    ax = axes[1, 1]
    cm = confusion_matrix(y, exemplo['av']['y_pred'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax, cbar=False,
                xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'], annot_kws={"size": 14, "weight": "bold"}
    ax.set_title(f'Matriz de confusão (semente {exemplo["semente"]}, F1 {exemplo["av"]["F1"]:.3f})', fontweight='bold')
    ax.set_ylabel('Real')
    ax.set_xlabel('Previsto')

    # 6. Fidelidade do SMPC
    ax = axes[1, 2]
    for semente, grupo in df_hist.groupby('semente'):
        ax.plot(grupo['rodada'], grupo['erro_max'], marker='o', alpha=0.6, label=f'Semente {semente}')
    ax.axhline(NUM_CLIENTES * 0.5 / SMPC_ESCALA, color='black', ls='--', label='Limite teórico (n · 2^-25)')
    ax.set_yscale('log')
    ax.set_xlabel('Rodada')
    ax.set_title('Erro da agregação SMPC vs FedAvg em claro', fontweight='bold')
    ax.legend(fontsize=8)

    plt.tight_layout()
    plt.show()

print("\n[SUCESSO] Teste 4 (v5) concluído!")
```

```
================================================================================
 BASE: PAYSIM | 5 sementes | 5 rodadas x 7 épocas | 3 geradores
================================================================================
Carregando '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/paysim.csv'...
[DADOS] Features: 14 | Treino: 990487 (fraudes: 4927) | Validação: 330163 (fraudes: 1643) | Teste: 330163 (fraudes: 1643)

--- Semente 42 ---
  Rodada 1/5 | PR-AUC val 0.8765 | treino 193.6s | SMPC 220 ms (erro máx. vs claro 8.8e-08)
  Rodada 2/5 | PR-AUC val 0.8944 | treino 194.8s | SMPC 212 ms (erro máx. vs claro 8.8e-08)
  Rodada 3/5 | PR-AUC val 0.9099 | treino 192.2s | SMPC 202 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9224 | treino 187.3s | SMPC 202 ms (erro máx. vs claro 8.8e-08)
  Rodada 5/5 | PR-AUC val 0.9280 | treino 187.1s | SMPC 223 ms (erro máx. vs claro 8.9e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9992 | PR-AUC 0.9309 | F1 0.7838

--- Semente 43 ---
  Rodada 1/5 | PR-AUC val 0.8681 | treino 191.5s | SMPC 199 ms (erro máx. vs claro 8.8e-08)
  Rodada 2/5 | PR-AUC val 0.8859 | treino 187.0s | SMPC 198 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8975 | treino 188.3s | SMPC 197 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9168 | treino 188.8s | SMPC 197 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9243 | treino 188.9s | SMPC 204 ms (erro máx. vs claro 8.9e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9992 | PR-AUC 0.9249 | F1 0.7760

--- Semente 44 ---
  Rodada 1/5 | PR-AUC val 0.8722 | treino 188.8s | SMPC 204 ms (erro máx. vs claro 8.8e-08)
  Rodada 2/5 | PR-AUC val 0.8926 | treino 185.4s | SMPC 197 ms (erro máx. vs claro 8.8e-08)
  Rodada 3/5 | PR-AUC val 0.9096 | treino 184.8s | SMPC 197 ms (erro máx. vs claro 8.8e-08)
  Rodada 4/5 | PR-AUC val 0.9204 | treino 184.9s | SMPC 239 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9244 | treino 186.4s | SMPC 223 ms (erro máx. vs claro 8.9e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9991 | PR-AUC 0.9294 | F1 0.7825

--- Semente 45 ---
  Rodada 1/5 | PR-AUC val 0.8664 | treino 188.1s | SMPC 216 ms (erro máx. vs claro 8.8e-08)
  Rodada 2/5 | PR-AUC val 0.8862 | treino 187.0s | SMPC 198 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.9024 | treino 191.5s | SMPC 206 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9180 | treino 192.5s | SMPC 216 ms (erro máx. vs claro 8.8e-08)
  Rodada 5/5 | PR-AUC val 0.9269 | treino 187.7s | SMPC 202 ms (erro máx. vs claro 8.8e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9992 | PR-AUC 0.9256 | F1 0.7787

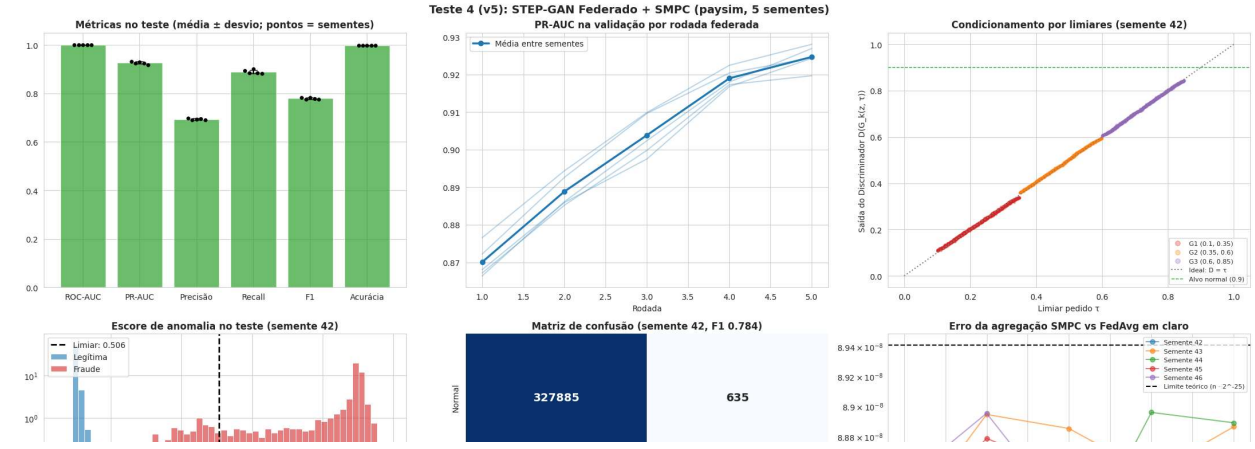
--- Semente 46 ---
  Rodada 1/5 | PR-AUC val 0.8672 | treino 187.6s | SMPC 203 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8852 | treino 188.5s | SMPC 230 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8998 | treino 186.6s | SMPC 200 ms (erro máx. vs claro 8.8e-08)
  Rodada 4/5 | PR-AUC val 0.9173 | treino 187.9s | SMPC 209 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9197 | treino 187.0s | SMPC 197 ms (erro máx. vs claro 8.8e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9992 | PR-AUC 0.9183 | F1 0.7752

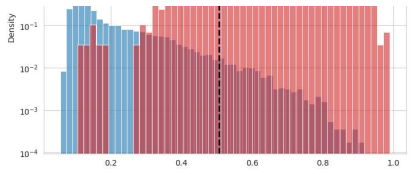
>> RESULTADOS POR SEMENTE (teste; limiar escolhido na validação):
         Rodada escolhida  ROC-AUC  PR-AUC  Precisão  Recall     F1  Acurácia       TPS  SMPC (ms/rodada) Erro máx. SMPC
Semente
42                      5   0.9992  0.9309    0.6981  0.8935 0.7838    0.9975 5,171,268          211.8203        8.9e-08
43                      5   0.9992  0.9249    0.6912  0.8844 0.7760    0.9975 5,182,787          199.0925        8.9e-08
44                      5   0.9991  0.9294    0.6921  0.9002 0.7825    0.9975 5,382,706          212.0089        8.9e-08
45                      5   0.9992  0.9256    0.6955  0.8844 0.7787    0.9975 5,149,043          207.6120        8.9e-08
46                      5   0.9992  0.9183    0.6911  0.8825 0.7752    0.9975 5,269,486          207.5191        8.9e-08

>> MÉDIA ± DESVIO-PADRÃO (5 sementes) e IC 95% (t de Student):
  ROC-AUC   : 0.9992 ± 0.0000   IC95% [0.9991, 0.9992]
  PR-AUC    : 0.9258 ± 0.0049   IC95% [0.9198, 0.9319]
  Precisão  : 0.6936 ± 0.0031   IC95% [0.6898, 0.6974]
  Recall    : 0.8890 ± 0.0076   IC95% [0.8796, 0.8984]
  F1        : 0.7792 ± 0.0038   IC95% [0.7745, 0.7840]
  Acurácia  : 0.9975 ± 0.0000   IC95% [0.9974, 0.9975]

>> CONDICIONAMENTO POR LIMIARES (semente 42): τ pedido × D(G_k(z, τ)) obtido
  G1 (0.1, 0.35): D médio 0.225 | erro absoluto médio 0.004 | correlação τ×D +0.999
  G2 (0.35, 0.6): D médio 0.477 | erro absoluto médio 0.004 | correlação τ×D +0.999
  G3 (0.6, 0.85): D médio 0.725 | erro absoluto médio 0.002 | correlação τ×D +0.999

>> SMPC: 208 ms por rodada | erro máx. vs FedAvg em claro 8.9e-08 | 17.4 MB enviados por cliente por rodada
Resultados por semente salvos em 'resultados_teste4_paysim.csv'.
```






```
================================================================================
 BASE: CREDITCARD | 5 sementes | 5 rodadas x 7 épocas | 3 geradores
================================================================================
Carregando '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/creditcard/creditcard.csv'...
[DADOS] Features: 29 | Treino: 59334 (fraudes: 295) | Validação: 19779 (fraudes: 99) | Teste: 19779 (fraudes: 98)

--- Semente 42 ---
  Rodada 1/5 | PR-AUC val 0.8092 | treino 11.3s | SMPC 212 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8250 | treino 11.5s | SMPC 252 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8315 | treino 11.8s | SMPC 250 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.8330 | treino 11.5s | SMPC 249 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.8354 | treino 11.3s | SMPC 253 ms (erro máx. vs claro 8.9e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9832 | PR-AUC 0.8564 | F1 0.7887

--- Semente 43 ---
  Rodada 1/5 | PR-AUC val 0.8039 | treino 11.4s | SMPC 209 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8220 | treino 11.3s | SMPC 208 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8291 | treino 11.7s | SMPC 219 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.8308 | treino 11.4s | SMPC 216 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.8322 | treino 11.3s | SMPC 206 ms (erro máx. vs claro 8.9e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9827 | PR-AUC 0.8536 | F1 0.7962

--- Semente 44 ---
  Rodada 1/5 | PR-AUC val 0.8202 | treino 11.4s | SMPC 207 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8315 | treino 11.4s | SMPC 210 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8335 | treino 11.4s | SMPC 211 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.8340 | treino 11.6s | SMPC 210 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.8342 | treino 11.6s | SMPC 206 ms (erro máx. vs claro 8.9e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9852 | PR-AUC 0.8528 | F1 0.7925

--- Semente 45 ---
  Rodada 1/5 | PR-AUC val 0.8105 | treino 11.3s | SMPC 209 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8227 | treino 11.4s | SMPC 210 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8293 | treino 11.6s | SMPC 208 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.8304 | treino 11.6s | SMPC 209 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.8309 | treino 11.5s | SMPC 213 ms (erro máx. vs claro 8.9e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9851 | PR-AUC 0.8558 | F1 0.8038

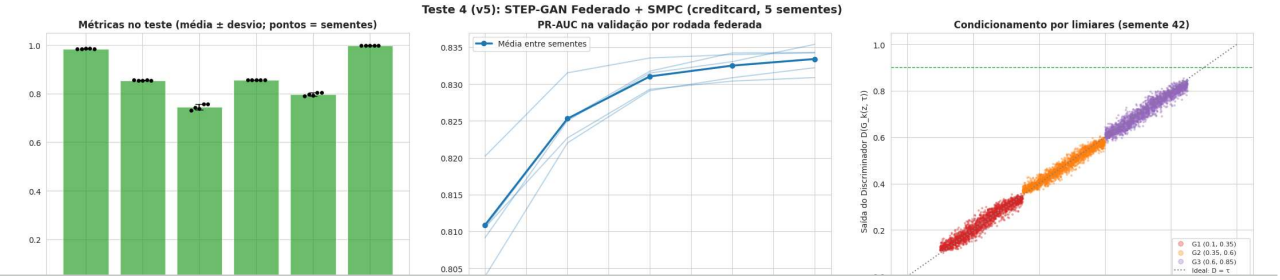
--- Semente 46 ---
  Rodada 1/5 | PR-AUC val 0.8106 | treino 11.4s | SMPC 252 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8253 | treino 11.5s | SMPC 240 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8317 | treino 11.5s | SMPC 246 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.8342 | treino 11.4s | SMPC 249 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.8343 | treino 11.5s | SMPC 220 ms (erro máx. vs claro 8.9e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9839 | PR-AUC 0.8540 | F1 0.8038

>> RESULTADOS POR SEMENTE (teste; limiar escolhido na validação):
         Rodada escolhida  ROC-AUC  PR-AUC  Precisão  Recall     F1  Acurácia       TPS  SMPC (ms/rodada) Erro máx. SMPC
Semente
42                      5   0.9832  0.8564    0.7304  0.8571 0.7887    0.9977 2,516,369          243.2824        8.9e-08
43                      5   0.9827  0.8536    0.7434  0.8571 0.7962    0.9978 2,773,608          211.7476        8.9e-08
44                      5   0.9852  0.8528    0.7368  0.8571 0.7925    0.9978 2,809,900          208.7380        8.9e-08
45                      5   0.9851  0.8558    0.7568  0.8571 0.8038    0.9979 2,752,596          209.8710        8.9e-08
46                      5   0.9839  0.8540    0.7568  0.8571 0.8038    0.9979 2,809,749          241.0963        8.9e-08

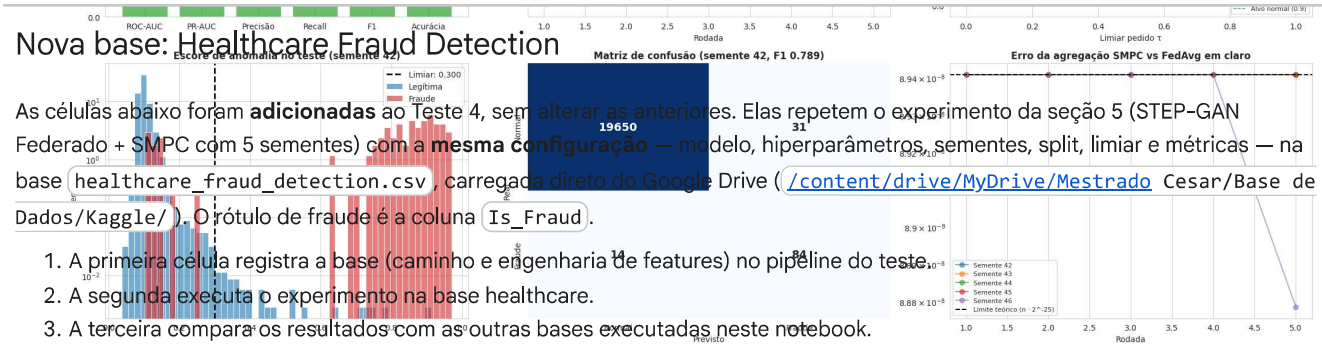
>> MÉDIA ± DESVIO-PADRÃO (5 sementes) e IC 95% (t de Student):
  ROC-AUC  : 0.9840 ± 0.0011   IC95% [0.9826, 0.9854]
  PR-AUC   : 0.8545 ± 0.0015   IC95% [0.8526, 0.8564]
  Precisão : 0.7448 ± 0.0118   IC95% [0.7302, 0.7595]
  Recall   : 0.8571 ± 0.0000   IC95% [0.8571, 0.8571]
  F1       : 0.7970 ± 0.0068   IC95% [0.7886, 0.8054]
  Acurácia : 0.9978 ± 0.0001   IC95% [0.9977, 0.9979]

>> CONDICIONAMENTO POR LIMIARES (semente 42): τ pedido × D(G_k(z, τ)) obtido
  G1 (0.1, 0.35): D médio 0.229 | erro absoluto médio 0.015 | correlação τ×D +0.965
  G2 (0.35, 0.6): D médio 0.476 | erro absoluto médio 0.014 | correlação τ×D +0.972
  G3 (0.6, 0.85): D médio 0.719 | erro absoluto médio 0.017 | correlação τ×D +0.962

>> SMPC: 223 ms por rodada | erro máx. vs FedAvg em claro 8.9e-08 | 18.2 MB enviados por cliente por rodada
Resultados por semente salvos em 'resultados_teste4_creditcard.csv'.
```



Nova base: Healthcare Fraud Detection



As células abaixo foram **adicionadas** ao Teste 4, sem alterar as anteriores. Elas repetem o experimento da seção 5 (STEP-GAN Federado + SMPC com 5 sementes) com a **mesma configuração** — modelo, hiperparâmetros, sementes, split, limiar e métricas — na base `healthcare_fraud_detection.csv`, carregada direto do Google Drive (`/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/`). O rótulo de fraude é a coluna `Is_Fraud`.

1. A primeira célula registra a base (caminho e engenharia de features) no pipeline do teste.
2. A segunda executa o experimento na base healthcare.
3. A terceira compara os resultados com as outras bases executadas neste notebook.

Engenharia de features da base healthcare (a mesma nos seis testes):

- removidos `Provider_ID` e `Claim_ID` (identificadores, como `nameOrig` / `nameDest` na PaySim) e `Claim_Submission_Date` (carimbo de tempo, como `Time` na Credit Card);
- `claim_amount_log` e `approved_amount_log` (log dos valores, como `amount_log`) e `approved_ratio` = valor aprovado ÷ valor solicitado (análoga ao `errorBalance` da PaySim);
- One-Hot das categóricas (`Patient_Gender`, `Diagnosis_Code`, `Procedure_Code`, `Insurance_Type`, `Provider_Specialty`, `Patient_State`, `Claim_Status`, `Visit_Type`), com ausentes na categoria `Desconhecido`;
- mediana para numéricas ausentes (`Prior_Visits_12m`).

```
# ======================================================================
# --- NOVA BASE: Healthcare Fraud Detection (mesma configuração do teste) ---
# ======================================================================
CAMINHOS['healthcare'] = [
    '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_fraud_detection.csv',
    'base_de_dados/healthcare_fraud_detection.csv',        # Cópia no repositório: executando de dentro de PoC/
    'PoC/base_de_dados/healthcare_fraud_detection.csv',    # Executando da raiz do repositório
]
COLUNAS_CATEGORICAS_HC = ['Patient_Gender', 'Diagnosis_Code', 'Procedure_Code', 'Insurance_Type',
                          'Provider_Specialty', 'Patient_State', 'Claim_Status', 'Visit_Type']


def preparar_healthcare(df):
    # Mesma amostragem das outras bases (todas as fraudes + até 200 normais por fraude);
    # como esta base tem 8,3% de fraude, ela é mantida inteira.
    df = amostrar(df.dropna(subset=['Is_Fraud']), 'Is_Fraud')
    df = df.copy()
    df['Is_Fraud'] = df['Is_Fraud'].astype(int)
    # Identificadores não generalizam (como nameOrig/nameDest); a data é um carimbo de tempo (como Time)
    df = df.drop(columns=['Provider_ID', 'Claim_ID', 'Claim_Submission_Date'], errors='ignore')
    # Log dos valores (como amount_log) e proporção aprovada pela operadora (análoga ao errorBalance da PaySim)
    df['approved_ratio'] = df['Approved_Amount'] / df['Claim_Amount'].replace(0, np.nan)
    df['claim_amount_log'] = np.log1p(df['Claim_Amount'].clip(lower=0))
    df['approved_amount_log'] = np.log1p(df['Approved_Amount'].clip(lower=0))
    df = df.drop(columns=['Claim_Amount', 'Approved_Amount'])
    # Categóricas -> One-Hot (Procedure_Code é código, não quantidade); ausente vira 'Desconhecido'
    for col in COLUNAS_CATEGORICAS_HC:
        df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str)
    df = pd.get_dummies(df, columns=COLUNAS_CATEGORICAS_HC, drop_first=True, dtype=float)
    # Numéricas ausentes (Prior_Visits_12m) -> mediana
    for col in df.select_dtypes(include=np.number).columns:
        if df[col].isnull().sum() > 0:
            df[col] = df[col].fillna(df[col].median())
    return df, 'Is_Fraud'


PREPARADORES['healthcare'] = preparar_healthcare
DATASETS_HEALTHCARE = ['healthcare']
print("Base healthcare registrada:", CAMINHOS['healthcare'][0])
```

```
Base healthcare registrada: /content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_fraud_detection.csv
```

```
# ======================================================================
# --- 5b. EXPERIMENTO NA BASE HEALTHCARE (mesma configuração da seção 5) ---
# ======================================================================

METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
resultados_gerais = globals().get('resultados_gerais', {})  # preserva os resultados das outras bases

for nome_base in DATASETS_HEALTHCARE:
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()} | {len(SEMENTES)} sementes | {NUM_RODADAS} rodadas x {EPOCAS_LOCAIS} épocas "
          f"| {len(DEGRAUS)} geradores")
    print("=" * 78)
```

```
        dados = carregar_dados(nome_base)
        if dados is None:
            print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_base]}. Pulando.")
            continue

        linhas, historicos, exemplo = [], [], None
        for semente in SEMENTES:
            print(f"\n--- Semente {semente} ---")
            estado_final, rodada_escolhida, historico = executar_federado(dados, semente)
            av = avaliar(estado_final['D'], dados)
            historico['semente'] = semente
            historicos.append(historico)
            linhas.append({'Semente': semente, 'Rodada escolhida': rodada_escolhida, **{m: av[m] for m in METRICAS},
                           'TPS': av['TPS'], 'SMPC (ms/rodada)': 1000 * historico['t_smpc'].mean(),
                           'Erro máx. SMPC': historico['erro_max'].max()})
            print(f"  -> Rodada escolhida: {rodada_escolhida} | ROC-AUC {av['ROC-AUC']:.4f} | PR-AUC {av['PR-AUC']:.4f} "
                  f"| F1 {av['F1']:.4f}")
            if exemplo is None:
                exemplo = dict(semente=semente, av=av,
                               aderencia=aderencia_condicionamento(estado_final, dados['input_dim']))

        df = pd.DataFrame(linhas).set_index('Semente')
        df_hist = pd.concat(historicos, ignore_index=True)

        print("\n>> RESULTADOS POR SEMENTE (teste; limiar escolhido na validação):")
        print(df.to_string(formatters={'Erro máx. SMPC': lambda x: f"{x:.1e}", 'TPS': lambda x: f"{x:,.0f}"},
                           float_format=lambda x: f"{x:.4f}"))

        print(f"\n>> MÉDIA ± DESVIO-PADRÃO ({len(SEMENTES)} sementes) e IC 95% (t de Student):")
        resumo = []
        for m in METRICAS:
            v = df[m].values
            dp = v.std(ddof=1) if len(v) > 1 else 0.0
            meia = stats.t.ppf(0.975, len(v) - 1) * dp / np.sqrt(len(v)) if len(v) > 1 else 0.0
            resumo.append({'Métrica': m, 'Média': v.mean(), 'Desvio-padrão': dp,
                           'IC95% inf': v.mean() - meia, 'IC95% sup': v.mean() + meia})
            print(f"  {m:9s}: {v.mean():.4f} ± {dp:.4f}   IC95% [{v.mean() - meia:.4f}, {v.mean() + meia:.4f}]")
        df_resumo = pd.DataFrame(resumo).set_index('Métrica')

        aderencia = exemplo['aderencia']
        print(f"\n>> CONDICIONAMENTO POR LIMIARES (semente {exemplo['semente']}): τ pedido × D(G_k(z, τ)) obtido")
        for g, grupo in aderencia.groupby('gerador'):
            corr = np.corrcoef(grupo['tau'], grupo['D'])[0, 1] if grupo['D'].std() > 0 else float('nan')
            print(f"  {g}: D médio {grupo['D'].mean():.3f} | erro absoluto médio {np.abs(grupo['D'] - grupo['tau']).mean():.3f
                  f"| correlação τ×D {corr:+.3f}")

        print(f"\n>> SMPC: {df['SMPC (ms/rodada)'].mean():.0f} ms por rodada | erro máx. vs FedAvg em claro "
              f"{df['Erro máx. SMPC'].max():.1e} | {df_hist['bytes_cliente'].iloc[0] / 1e6:.1f} MB enviados por cliente por roc

        arquivo = f'resultados_teste4_{nome_base}.csv'
        df.to_csv(arquivo)
        print(f"Resultados por semente salvos em '{arquivo}'.")
        resultados_gerais[nome_base] = dict(por_semente=df, resumo=df_resumo, historico=df_hist, aderencia=aderencia)

        # ----- Visualizações -----
        fig, axes = plt.subplots(2, 3, figsize=(22, 11))
        fig.suptitle(f"Teste 4 (v5): STEP-GAN Federado + SMPC ({nome_base}, {len(SEMENTES)} sementes)",
                     fontsize=14, fontweight='bold')

        # 1. Métricas: média ± desvio, com os pontos de cada semente
        ax = axes[0, 0]
        ax.bar(METRICAS, df_resumo['Média'], yerr=df_resumo['Desvio-padrão'], capsize=5, color='#2ca02c', alpha=0.7)
        for i, m in enumerate(METRICAS):
            ax.scatter(np.full(len(df), i) + np.linspace(-0.15, 0.15, len(df)), df[m], color='black', s=15, zorder=3)
        ax.set_ylim(0, 1.05)
        ax.set_title('Métricas no teste (média ± desvio; pontos = sementes)', fontweight='bold')

        # 2. Convergência do FL
        ax = axes[0, 1]
        for semente, grupo in df_hist.groupby('semente'):
            ax.plot(grupo['rodada'], grupo['pr_auc_val'], color='#1f77b4', alpha=0.3)
        media = df_hist.groupby('rodada')['pr_auc_val'].mean()
        ax.plot(media.index, media.values, color='#1f77b4', lw=2.5, marker='o', label='Média entre sementes')
        ax.set_title('PR-AUC na validação por rodada federada', fontweight='bold')
        ax.set_xlabel('Rodada')
        ax.legend()

        # 3. Condicionamento por limiares
        ax = axes[0, 2]
        cores_g = ['#d62728', '#ff7f0e', '#9467bd']
        for (g, grupo), cor in zip(aderencia.groupby('gerador'), cores_g):
            ax.scatter(grupo['tau'], grupo['D'], s=4, alpha=0.3, color=cor, label=g)
```

```
        ax.plot([0, 1], [0, 1], ':', color='gray', label='Ideal: D = τ')
        ax.axhline(ALVO_NORMAL, color='#2ca02c', ls='--', lw=1, label='Alvo normal (0.9)')
        ax.set_xlabel('Limiar pedido τ')
        ax.set_ylabel('Saída do Discriminador D(G_k(z, τ))')
        ax.set_title(f'Condicionamento por limiares (semente {exemplo["semente"]})', fontweight='bold')
        ax.legend(fontsize=8, markerscale=3)

        # 4. Distribuição dos escores
        ax = axes[1, 0]
        s, y = exemplo['av']['scores'], dados['y_test']
        sns.histplot(s[y == 0], color='#1f77b4', bins=50, alpha=0.6, label='Legítima', ax=ax, stat='density')
        sns.histplot(s[y == 1], color='#d62728', bins=50, alpha=0.6, label='Fraude', ax=ax, stat='density')
        ax.axvline(exemplo['av']['limiar'], color='black', ls='--', lw=2, label=f"Limiar: {exemplo['av']['limiar']:.3f}")
        ax.set_yscale('log')
        ax.set_title(f'Escore de anomalia no teste (semente {exemplo["semente"]})', fontweight='bold')
        ax.legend()

        # 5. Matriz de confusão
        ax = axes[1, 1]
        cm = confusion_matrix(y, exemplo['av']['y_pred'])
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax, cbar=False,
                    xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'], annot_kws={"size": 14, "weight": "bold'
        ax.set_title(f'Matriz de confusão (semente {exemplo["semente"]}, F1 {exemplo["av"]["F1"]:.3f})', fontweight='bold')
        ax.set_ylabel('Real')
        ax.set_xlabel('Previsto')

        # 6. Fidelidade do SMPC
        ax = axes[1, 2]
        for semente, grupo in df_hist.groupby('semente'):
            ax.plot(grupo['rodada'], grupo['erro_max'], marker='o', alpha=0.6, label=f'Semente {semente}')
        ax.axhline(NUM_CLIENTES * 0.5 / SMPC_ESCALA, color='black', ls='--', label='Limite teórico (n · 2^-25)')
        ax.set_yscale('log')
        ax.set_xlabel('Rodada')
        ax.set_title('Erro da agregação SMPC vs FedAvg em claro', fontweight='bold')
        ax.legend(fontsize=8)

        plt.tight_layout()
        plt.show()

print("\n[SUCESSO] Teste 4 (v5) concluído na base healthcare!")
```

```
================================================================================
 BASE: HEALTHCARE | 5 sementes | 5 rodadas x 7 épocas | 3 geradores
================================================================================
Carregando '/content/drive/MyDrive/Mestrado Cesar/Base de Dados/Kaggle/healthcare_fraud_detection.csv'...
[DADOS] Features: 48 | Treino: 6000 (fraudes: 497) | Validação: 2000 (fraudes: 166) | Teste: 2000 (fraudes: 166)

--- Semente 42 ---
  Rodada 1/5 | PR-AUC val 0.7073 | treino 1.2s | SMPC 230 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.8326 | treino 1.5s | SMPC 235 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.9088 | treino 1.2s | SMPC 218 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9436 | treino 1.2s | SMPC 222 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9620 | treino 1.2s | SMPC 234 ms (erro máx. vs claro 8.9e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.9965 | PR-AUC 0.9727 | F1 0.8122

--- Semente 43 ---
  Rodada 1/5 | PR-AUC val 0.6179 | treino 1.2s | SMPC 219 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.7657 | treino 1.2s | SMPC 247 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.8848 | treino 1.3s | SMPC 292 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.9351 | treino 1.3s | SMPC 230 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.9568 | treino 1.4s | SMPC 217 ms (erro máx. vs claro 8.9e-08)
```

```
# ======================================================================
# --- COMPARATIVO ENTRE AS BASES (Teste 4: STEP-GAN Federado + SMPC) ---
# ======================================================================
import contextlib
import io

NOMES_BASES = {'creditcard': 'Credit Card', 'paysim': 'PaySim', 'healthcare': 'Healthcare'}
# Cor fixa por base (paleta categórica validada para daltonismo): a mesma base tem sempre a mesma cor
CORES_BASES = {'creditcard': '#2a78d6', 'healthcare': '#eb6834', 'paysim': '#1baf7a'}
bases = [b for b in NOMES_BASES if b in globals().get('resultados_gerais', {})]


def perfil_da_base(nome):
    """Tamanho e taxa de fraude do conjunto de teste (a base é recarregada só para essas estatísticas)."""
    with contextlib.redirect_stdout(io.StringIO()):
        d = carregar_dados(nome)
    return {'Transações (teste)': len(d['y_test']), 'Taxa de fraude (teste)': float(np.mean(d['y_test'])),
            'Features': int(d['input_dim'])}

if not bases:
    print("Nenhum resultado disponível: execute a seção 5 e/ou a base healthcare antes do comparativo.")
else:
    perfis = {b: perfil_da_base(b) for b in bases}
    linhas = []
    for b in bases:
        por_semente, aderencia_b = resultados_gerais[b]['por_semente'], resultados_gerais[b]['aderencia']
        taxa = perfis[b]['Taxa de fraude (teste)']
        linha = {'Base': NOMES_BASES[b],
                 'Transações (teste)': f"{perfis[b]['Transações (teste)']:,}".replace(',', '.'),
                 'Taxa de fraude (teste)': f"{taxa:.2%}", 'Features': perfis[b]['Features']}
        for m in METRICAS:
            linha[m] = f"{por_semente[m].mean():.4f} ± {por_semente[m].std(ddof=1):.4f}"
        linha['PR-AUC normalizado'] = f"{(por_semente['PR-AUC'].mean() - taxa) / (1 - taxa):.4f}"
        correlacoes = [np.corrcoef(g['tau'], g['D'])[0, 1] for _, g in aderencia_b.groupby('gerador')]
        linha['Correlação τ×D (média dos geradores)'] = f"{np.mean(correlacoes):.3f}"
        linha['Rodada escolhida (mais frequente)'] = int(por_semente['Rodada escolhida'].mode().iloc[0])
        linha['Erro máx. SMPC'] = f"{por_semente['Erro máx. SMPC'].max():.1e}"
        linhas.append(linha)
    tabela_comparativa = pd.DataFrame(linhas).set_index('Base')
    print(f">> COMPARATIVO ENTRE AS BASES — Teste 4 ({len(SEMENTES)} sementes; média ± desvio-padrão no teste)")
    print(tabela_comparativa.T.to_string())

    if {'creditcard', 'healthcare'} <= set(bases):
        cc, hc = resultados_gerais['creditcard']['por_semente'], resultados_gerais['healthcare']['por_semente']
        print("\n>> DIFERENÇA DAS MÉDIAS (Healthcare − Credit Card)")
        for m in METRICAS:
            print(f"  {m:9s}: {hc[m].mean() - cc[m].mean():+.4f}")
    print("\nObservação: o ROC-AUC não depende da taxa de fraude e é a métrica mais direta para comparar as bases. "
          "O PR-AUC depende dela (um classificador aleatório tem PR-AUC igual à taxa de fraude); por isso a comparação "
          "do PR-AUC entre bases deve usar o PR-AUC normalizado = (PR-AUC − taxa) / (1 − taxa): 0 = aleatório, 1 = perfeito

    fig, axes = plt.subplots(1, 3, figsize=(22, 5.5))
    largura = 0.8 / len(bases)
    x = np.arange(len(METRICAS))
    for i, b in enumerate(bases):
        por_semente = resultados_gerais[b]['por_semente']
        axes[0].bar(x + (i - (len(bases) - 1) / 2) * largura, [por_semente[m].mean() for m in METRICAS], largura,
                    yerr=[por_semente[m].std(ddof=1) for m in METRICAS], capsize=3,
                    color=CORES_BASES[b], label=NOMES_BASES[b], edgecolor='white', linewidth=1.5)
    axes[0].set_xticks(x)
    axes[0].set_xticklabels(METRICAS)
    axes[0].set_ylim(0, 1.2)
```

```
    axes[0].set_title('Métricas no teste (média ± desvio)', fontweight='bold')
    axes[0].legend(loc='upper center', ncol=len(bases), frameon=False)

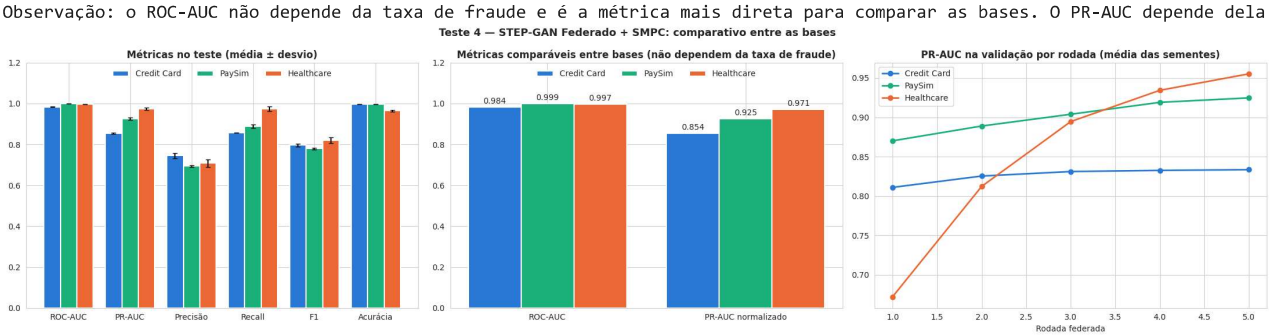
    comparaveis = ['ROC-AUC', 'PR-AUC normalizado']
    x2 = np.arange(len(comparaveis))
    for i, b in enumerate(bases):
        taxa = perfis[b]['Taxa de fraude (teste)']
        pr_normalizado = (resultados_gerais[b]['por_semente']['PR-AUC'].mean() - taxa) / (1 - taxa)
        valores = [resultados_gerais[b]['por_semente']['ROC-AUC'].mean(), pr_normalizado]
        barras = axes[1].bar(x2 + (i - (len(bases) - 1) / 2) * largura, valores, largura, color=CORES_BASES[b],
                             label=NOMES_BASES[b], edgecolor='white', linewidth=1.5)
        axes[1].bar_label(barras, fmt='%.3f', padding=2)
    axes[1].set_xticks(x2)
    axes[1].set_xticklabels(comparaveis)
    axes[1].set_ylim(0, 1.2)
    axes[1].set_title('Métricas comparáveis entre bases (não dependem da taxa de fraude)', fontweight='bold')
    axes[1].legend(loc='upper center', ncol=len(bases), frameon=False)

    for b in bases:
        media = resultados_gerais[b]['historico'].groupby('rodada')['pr_auc_val'].mean()
        axes[2].plot(media.index, media.values, marker='o', lw=2, color=CORES_BASES[b], label=NOMES_BASES[b])
    axes[2].set_xlabel('Rodada federada')
    axes[2].set_title('PR-AUC na validação por rodada (média das sementes)', fontweight='bold')
    axes[2].legend()
    plt.suptitle('Teste 4 — STEP-GAN Federado + SMPC: comparativo entre as bases', fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.show()
```

```
>> COMPARATIVO ENTRE AS BASES — Teste 4 (5 sementes; média ± desvio-padrão no teste)
Base                                  Credit Card           PaySim       Healthcare
Transações (teste)                         19.779           330.163            2.000
Taxa de fraude (teste)                      0.50%             0.50%            8.30%
Features                                       29                14               48
ROC-AUC                          0.9840 ± 0.0011  0.9992 ± 0.0000  0.9968 ± 0.0010
PR-AUC                           0.8545 ± 0.0015  0.9258 ± 0.0049  0.9738 ± 0.0063
Precisão                         0.7448 ± 0.0118  0.6936 ± 0.0031  0.7079 ± 0.0195
Recall                           0.8571 ± 0.0000  0.8890 ± 0.0076  0.9735 ± 0.0109
F1                               0.7970 ± 0.0068  0.7792 ± 0.0038  0.8196 ± 0.0140
Acurácia                         0.9978 ± 0.0001  0.9975 ± 0.0000  0.9644 ± 0.0032
PR-AUC normalizado                         0.8538            0.9255           0.9714
Correlação τ×D (média dos geradores)        0.966             0.999            0.084
Rodada escolhida (mais frequente)               5                 5                5
Erro máx. SMPC                            8.9e-08           8.9e-08          8.9e-08

>> DIFERENÇA DAS MÉDIAS (Healthcare − Credit Card)
  ROC-AUC   : +0.0128
  PR-AUC    : +0.1193
  Precisão  : -0.0369
  Recall    : +0.1164
  F1        : +0.0226
  Acurácia  : -0.0334

Observação: o ROC-AUC não depende da taxa de fraude e é a métrica mais direta para comparar as bases. O PR-AUC depende dela
```



Novas bases: Bank Account Fraud (NeurIPS 2022), Insurance Claims e Fraud Oracle

As células abaixo foram **adicionadas** ao Teste 4, sem alterar as anteriores. Elas repetem o experimento da seção 5 (STEP-GAN Federado + SMPC com 5 sementes) com a **mesma configuração** — modelo, hiperparâmetros, sementes, split, limiar e métricas — em mais três bases.

Origem das bases:

- **Bank Account Fraud Dataset Suite (NeurIPS 2022)** — só o arquivo `Base.csv` (213 MB, 1 milhão de solicitações de abertura de conta, ~1,1% de fraude), baixado com `kagglehub` (já instalado no Colab; localmente, `pip install kagglehub`): `kagglehub.dataset_download("sgpjesus/bank-account-fraud-dataset-neurips-2022", path="Base.csv")`. Sem acesso ao Kaggle, é usada a cópia do repositório, `PoC/base_de_dados/Base.csv.zip`.
- **Insurance Claims** — `insurance_claims.csv` (1.000 sinistros de seguro de automóvel, 24,7% de fraude), no repositório GitHub do projeto, em `PoC/base_de_dados/insurance_claims.csv`: a célula usa a cópia local quando o notebook roda a partir do repositório e, no Colab, lê o arquivo direto do GitHub.
- **Fraud Oracle** — `fraud_oracle.csv` (15.420 sinistros de seguro de veículos, 6,0% de fraude; rótulo `FraudFound_P`: 1 = fraude), no repositório GitHub do projeto, em `PoC/base_de_dados/fraud_oracle.csv`, lida da mesma forma que a Insurance Claims.

1. A primeira célula registra as três bases (origem e engenharia de features) no pipeline do teste.
2. A segunda executa o experimento na base BAF (seção 5c).
3. A terceira executa o experimento na base Insurance Claims (seção 5d).
4. A quarta executa o experimento na base Fraud Oracle (seção 5e).
5. A quinta compara os resultados de todas as bases executadas neste notebook (Credit Card, Healthcare, BAF, Insurance Claims e Fraud Oracle).

Tamanho das bases: a amostragem é a mesma das outras bases (todas as fraudes + até 200 normais por fraude), e as três entram inteiras. A BAF (1 milhão de linhas, no porte da PaySim amostrada) deixa o experimento bem mais demorado que na Credit Card; para uma execução exploratória mais rápida, `AMOSTRA_MAXIMA_NOVAS_BASES` (célula de registro) limita cada base a uma amostra estratificada, e o padrão `None` mantém a configuração do teste. A Insurance Claims é pequena (1.000 linhas, ~200 de treino por cliente): os resultados dela oscilam mais entre sementes e o IC fica mais largo. A Fraud Oracle (15.420 linhas) roda em poucos minutos.

Engenharia de features (a mesma nos seis testes):

- **BAF** (rótulo `fraud_bool`): removidos `month` (mês da solicitação, carimbo de tempo como `Time` na Credit Card) e `device_fraud_count` (constante na `Base.csv`); nas seis colunas em que valor negativo significa ausente (`prev_address_months_count`, `current_address_months_count`, `intended_balcon_amount`, `bank_months_count`, `session_length_in_minutes`, `device_distinct_emails_8w`), criado um indicador `_ausente` e o valor ausente vai para a mediana; log de `intended_balcon_amount`, `days_since_request`, `zip_count_4w` e `proposed_credit_limit`; One-Hot de `payment_type`, `employment_status`, `housing_status`, `source` e `device_os`.
- **Insurance Claims** (rótulo `fraud_reported`, Y/N → 1/0): removidos `policy_number`, `insured_zip` e `incident_location` (identificadores, como `nameOrig`/`nameDest` na PaySim), `_c39` (coluna vazia) e `auto_model` (39 modelos para 1.000 linhas; `auto_make` é mantida); as datas `policy_bind_date` e `incident_date` viram `dias_apolice_ate_incidente`; log de `total_claim_amount`, `injury_claim`, `property_claim` e `vehicle_claim`; One-Hot das 16 categóricas, com `?` na categoria `Desconhecido` e `authorities_contacted` ausente (texto `None` no CSV) como `Nenhuma`.
- **Fraud Oracle** (rótulo `FraudFound_P`): removidos `PolicyNumber` e `RepNumber` (identificadores), `Year` (carimbo de tempo, como `Time`) e `PolicyType` (repete `VehicleCategory` + `BasePolicy`); `Month` e `MonthClaimed` viram `meses_ate_sinistro`; as nove colunas de faixas em texto (`VehiclePrice`, `Days_Policy_Accident`, `Days_Policy_Claim`, `PastNumberOfClaims`, `AgeOfVehicle`, `AgeOfPolicyHolder`, `NumberOfSuppliments`, `AddressChange_Claim`, `NumberOfCars`) viram números em ordem crescente; `Age` = 0 (inválido) vai para a mediana; One-Hot das 12 categóricas, com `0` em `DayOfWeekClaimed` como `Desconhecido`.

```
# ======================================================================
# --- NOVAS BASES: Bank Account Fraud (Base.csv), Insurance Claims e Fraud Oracle (mesma configuração do teste) ---
# ======================================================================
AMOSTRA_MAXIMA_NOVAS_BASES = None   # None = base inteira (configuração do teste); ex.: 300_000 para
                                    # uma execução exploratória mais rápida (amostra estratificada)


def baixar_base_baf():
    """Baixa só o arquivo Base.csv com kagglehub (None se o Kaggle não estiver acessível)."""
    try:
        import kagglehub
        # Download latest version — apenas o arquivo Base.csv (213 MB)
        path = kagglehub.dataset_download("sgpjesus/bank-account-fraud-dataset-neurips-2022", path="Base.csv")
        print("Path to dataset files:", path)
        return path
    except Exception as erro:
        print(f"Download pelo kagglehub indisponível ({type(erro).__name__}): usando a cópia do repositório.")
        return None


CAMINHOS['baf'] = [c for c in [
    baixar_base_baf(),
    'base_de_dados/Base.csv.zip',      # Cópia no repositório (zip lido direto pelo pandas): de dentro de PoC/
    'PoC/base_de_dados/Base.csv.zip',  # Executando da raiz do repositório
    'base_de_dados/Base.csv',
    'PoC/base_de_dados/Base.csv',
```

```
] if c]
# Insurance Claims: arquivo no repositório GitHub do projeto (PoC/base_de_dados/insurance_claims.csv)
CAMINHOS['insurance'] = [
    'base_de_dados/insurance_claims.csv',        # Executando de dentro de PoC/
    'PoC/base_de_dados/insurance_claims.csv',    # Executando da raiz do repositório
]
if not any(os.path.exists(c) for c in CAMINHOS['insurance']):
    # No Colab (sem o repositório clonado), baixa o arquivo direto do GitHub
    import urllib.request
    try:
        urllib.request.urlretrieve('https://raw.githubusercontent.com/suxlcmd/MPES_Lucas_Cerqueira/main/PoC/base_de_dados/i
        CAMINHOS['insurance'].append('insurance_claims.csv')
        print("insurance_claims.csv baixado do repositório GitHub.")
    except Exception as erro:
        print(f"Não foi possível baixar insurance_claims.csv do GitHub ({type(erro).__name__}).")
# Fraud Oracle: arquivo no repositório GitHub do projeto (PoC/base_de_dados/fraud_oracle.csv)
CAMINHOS['oracle'] = [
    'base_de_dados/fraud_oracle.csv',        # Executando de dentro de PoC/
    'PoC/base_de_dados/fraud_oracle.csv',    # Executando da raiz do repositório
]
if not any(os.path.exists(c) for c in CAMINHOS['oracle']):
    # No Colab (sem o repositório clonado), baixa o arquivo direto do GitHub
    import urllib.request
    try:
        urllib.request.urlretrieve('https://raw.githubusercontent.com/suxlcmd/MPES_Lucas_Cerqueira/main/PoC/base_de_dados/f
        CAMINHOS['oracle'].append('fraud_oracle.csv')
        print("fraud_oracle.csv baixado do repositório GitHub.")
    except Exception as erro:
        print(f"Não foi possível baixar fraud_oracle.csv do GitHub ({type(erro).__name__}).")


def reduzir(df, alvo):
    """Mesma amostragem das outras bases (todas as fraudes + até 200 normais por fraude) e, opcionalmente,
    uma amostra estratificada de AMOSTRA_MAXIMA_NOVAS_BASES linhas."""
    df = amostrar(df.dropna(subset=[alvo]), alvo)
    if AMOSTRA_MAXIMA_NOVAS_BASES and len(df) > AMOSTRA_MAXIMA_NOVAS_BASES:
        print(f"Amostra estratificada de {AMOSTRA_MAXIMA_NOVAS_BASES:,} linhas (preserva a taxa de fraude)...")
        df, _ = train_test_split(df, train_size=AMOSTRA_MAXIMA_NOVAS_BASES, stratify=df[alvo], random_state=42)
    return df.copy()


def preparar_baf(df):
    df = reduzir(df, 'fraud_bool')
    df['fraud_bool'] = df['fraud_bool'].astype(int)
    # month: carimbo de tempo (como Time); device_fraud_count: constante (zero) na Base.csv
    df = df.drop(columns=['month', 'device_fraud_count'], errors='ignore')
    # Valor negativo nestas colunas codifica "ausente" (documentação do dataset): indicador + mediana
    for col in ['prev_address_months_count', 'current_address_months_count', 'intended_balcon_amount',
                'bank_months_count', 'session_length_in_minutes', 'device_distinct_emails_8w']:
        df[col + '_ausente'] = (df[col] < 0).astype(float)
        df[col] = df[col].where(df[col] >= 0)
    # Contagens e valores com cauda longa -> log (como amount_log nas outras bases)
    for col in ['intended_balcon_amount', 'days_since_request', 'zip_count_4w', 'proposed_credit_limit']:
        df[col + '_log'] = np.log1p(df[col].clip(lower=0))
    df = df.drop(columns=['intended_balcon_amount', 'days_since_request', 'zip_count_4w', 'proposed_credit_limit'])
    categoricas = ['payment_type', 'employment_status', 'housing_status', 'source', 'device_os']
    for col in categoricas:
        df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str)
    df = pd.get_dummies(df, columns=categoricas, drop_first=True, dtype=float)
    for col in df.select_dtypes(include=np.number).columns:
        if df[col].isnull().sum() > 0:
            df[col] = df[col].fillna(df[col].median())
    return df, 'fraud_bool'


def preparar_insurance(df):
    df = df.copy()
    df['fraud_reported'] = df['fraud_reported'].astype(str).str.strip().str.upper().map({'Y': 1, 'N': 0})
    df = reduzir(df, 'fraud_reported')
    df['fraud_reported'] = df['fraud_reported'].astype(int)
    # Datas -> um único intervalo (dias entre a contratação da apólice e o incidente)
    df['dias_apolice_ate_incidente'] = (pd.to_datetime(df['incident_date'], errors='coerce')
                                        - pd.to_datetime(df['policy_bind_date'], errors='coerce')).dt.days
    # Identificadores (como nameOrig/nameDest), coluna vazia, auto_model (39 modelos em 1.000 linhas) e as datas
    df = df.drop(columns=['policy_number', 'insured_zip', 'incident_location', '_c39', 'auto_model',
                          'policy_bind_date', 'incident_date'], errors='ignore')
    # Valores do sinistro com cauda longa -> log (como amount_log nas outras bases)
    for col in ['total_claim_amount', 'injury_claim', 'property_claim', 'vehicle_claim']:
        df[col + '_log'] = np.log1p(df[col].clip(lower=0))
    df = df.drop(columns=['total_claim_amount', 'injury_claim', 'property_claim', 'vehicle_claim'])
    # '?' = desconhecido; authorities_contacted 'None' (lido como ausente) = nenhuma autoridade acionada
    df['authorities_contacted'] = df['authorities_contacted'].fillna('Nenhuma')
```

```
    categoricas = ['policy_state', 'policy_csl', 'insured_sex', 'insured_education_level', 'insured_occupation',
                   'insured_hobbies', 'insured_relationship', 'incident_type', 'collision_type', 'incident_severity',
                   'authorities_contacted', 'incident_state', 'incident_city', 'property_damage',
                   'police_report_available', 'auto_make']
    for col in categoricas:
        df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str).replace('?', 'Desconhecido')
    df = pd.get_dummies(df, columns=categoricas, drop_first=True, dtype=float)
    for col in df.select_dtypes(include=np.number).columns:
        if df[col].isnull().sum() > 0:
            df[col] = df[col].fillna(df[col].median())
    return df, 'fraud_reported'


def preparar_oracle(df):
    df = reduzir(df, 'FraudFound_P')
    df['FraudFound_P'] = df['FraudFound_P'].astype(int)
    # PolicyNumber (sequencial) e RepNumber (representante) são identificadores, como nameOrig/nameDest na PaySim;
    # Year é carimbo de tempo, como 'Time' na base Credit Card; PolicyType repete VehicleCategory + BasePolicy.
    # Month e MonthClaimed viram um único intervalo: meses entre o acidente e a abertura do sinistro.
    MESES = {'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 'Jun': 6, 'Jul': 7, 'Aug': 8, 'Sep': 9, 'Oct': 10, 'Nov': 1
    df['meses_ate_sinistro'] = (df['MonthClaimed'].map(MESES) - df['Month'].map(MESES)) % 12
    colunas_remover = ['PolicyNumber', 'RepNumber', 'Year', 'PolicyType', 'Month', 'MonthClaimed']
    df = df.drop(columns=[col for col in colunas_remover if col in df.columns])
    # Faixas de texto -> números em ordem crescente (preserva a ordem e evita dezenas de colunas One-Hot)
    FAIXAS_ORDINAIS = {
        'VehiclePrice': ['less than 20000', '20000 to 29000', '30000 to 39000', '40000 to 59000', '60000 to 69000', 'more t
        'Days_Policy_Accident': ['none', '1 to 7', '8 to 15', '15 to 30', 'more than 30'],
        'Days_Policy_Claim': ['none', '8 to 15', '15 to 30', 'more than 30'],
        'PastNumberOfClaims': ['none', '1', '2 to 4', 'more than 4'],
        'AgeOfVehicle': ['new', '2 years', '3 years', '4 years', '5 years', '6 years', '7 years', 'more than 7'],
        'AgeOfPolicyHolder': ['16 to 17', '18 to 20', '21 to 25', '26 to 30', '31 to 35', '36 to 40', '41 to 50', '51 to 6
        'NumberOfSuppliments': ['none', '1 to 2', '3 to 5', 'more than 5'],
        'AddressChange_Claim': ['under 6 months', '1 year', '2 to 3 years', '4 to 8 years', 'no change'],
        'NumberOfCars': ['1 vehicle', '2 vehicles', '3 to 4', '5 to 8', 'more than 8'],
    }
    for col, ordem in FAIXAS_ORDINAIS.items():
        df[col] = df[col].astype(str).str.strip().map({faixa: i for i, faixa in enumerate(ordem)})
    # Age = 0 é um valor inválido na base (o motorista mais novo está na faixa 16 a 17 anos): tratado como ausente
    df['Age'] = df['Age'].where(df['Age'] > 0)

    # Categóricas -> One-Hot ('0' em DayOfWeekClaimed é valor desconhecido)
    colunas_categoricas = ['DayOfWeek', 'DayOfWeekClaimed', 'Make', 'AccidentArea', 'Sex', 'MaritalStatus', 'Fault',
                           'VehicleCategory', 'PoliceReportFiled', 'WitnessPresent', 'AgentType', 'BasePolicy']
    for col in colunas_categoricas:
        df[col] = df[col].astype('object').where(df[col].notna(), 'Desconhecido').astype(str).replace('0', 'Desconhecido')
    df = pd.get_dummies(df, columns=colunas_categoricas, drop_first=True, dtype=float)

    # Numéricas ausentes (faixas fora da lista, Age = 0, mês desconhecido) -> mediana
    for col in df.select_dtypes(include=np.number).columns:
        if df[col].isnull().sum() > 0:
            df[col] = df[col].fillna(df[col].median())
    return df, 'FraudFound_P'


PREPARADORES['baf'] = preparar_baf
PREPARADORES['insurance'] = preparar_insurance
PREPARADORES['oracle'] = preparar_oracle
DATASETS_BAF = ['baf']
DATASETS_INSURANCE = ['insurance']
DATASETS_ORACLE = ['oracle']
for nome_base in ['baf', 'insurance', 'oracle']:
    print(f"Base {nome_base} registrada; caminhos procurados: {CAMINHOS[nome_base]}")
```

```
Downloading to /root/.cache/kagglehub/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022/versions/2/Base.csv...
100%|██████████| 82.4M/82.4M [00:04<00:00, 17.4MB/s]Extracting zip of Base.csv...

Path to dataset files: /root/.cache/kagglehub/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022/versions/2/Base.csv
insurance_claims.csv baixado do repositório GitHub.
fraud_oracle.csv baixado do repositório GitHub.
Base baf registrada; caminhos procurados: ['/root/.cache/kagglehub/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022
Base insurance registrada; caminhos procurados: ['base_de_dados/insurance_claims.csv', 'PoC/base_de_dados/insurance_claims.c
Base oracle registrada; caminhos procurados: ['base_de_dados/fraud_oracle.csv', 'PoC/base_de_dados/fraud_oracle.csv', 'fraud
```

```
# ======================================================================
# --- 5c. EXPERIMENTO NA BASE BAF (mesma configuração da seção 5) ---
# ======================================================================

METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
resultados_gerais = globals().get('resultados_gerais', {})  # preserva os resultados das outras bases

for nome_base in DATASETS_BAF:
```

```
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()} | {len(SEMENTES)} sementes | {NUM_RODADAS} rodadas x {EPOCAS_LOCAIS} épocas "
          f"| {len(DEGRAUS)} geradores")
    print("=" * 78)
    dados = carregar_dados(nome_base)
    if dados is None:
        print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_base]}. Pulando.")
        continue

    linhas, historicos, exemplo = [], [], None
    for semente in SEMENTES:
        print(f"\n--- Semente {semente} ---")
        estado_final, rodada_escolhida, historico = executar_federado(dados, semente)
        av = avaliar(estado_final['D'], dados)
        historico['semente'] = semente
        historicos.append(historico)
        linhas.append({'Semente': semente, 'Rodada escolhida': rodada_escolhida, **{m: av[m] for m in METRICAS},
                       'TPS': av['TPS'], 'SMPC (ms/rodada)': 1000 * historico['t_smpc'].mean(),
                       'Erro máx. SMPC': historico['erro_max'].max()})
        print(f"  -> Rodada escolhida: {rodada_escolhida} | ROC-AUC {av['ROC-AUC']:.4f} | PR-AUC {av['PR-AUC']:.4f} "
              f"| F1 {av['F1']:.4f}")
        if exemplo is None:
            exemplo = dict(semente=semente, av=av,
                           aderencia=aderencia_condicionamento(estado_final, dados['input_dim']))

    df = pd.DataFrame(linhas).set_index('Semente')
    df_hist = pd.concat(historicos, ignore_index=True)

    print("\n>> RESULTADOS POR SEMENTE (teste; limiar escolhido na validação):")
    print(df.to_string(formatters={'Erro máx. SMPC': lambda x: f"{x:.1e}", 'TPS': lambda x: f"{x:,.0f}"},
                       float_format=lambda x: f"{x:.4f}"))

    print(f"\n>> MÉDIA ± DESVIO-PADRÃO ({len(SEMENTES)} sementes) e IC 95% (t de Student):")
    resumo = []
    for m in METRICAS:
        v = df[m].values
        dp = v.std(ddof=1) if len(v) > 1 else 0.0
        meia = stats.t.ppf(0.975, len(v) - 1) * dp / np.sqrt(len(v)) if len(v) > 1 else 0.0
        resumo.append({'Métrica': m, 'Média': v.mean(), 'Desvio-padrão': dp,
                       'IC95% inf': v.mean() - meia, 'IC95% sup': v.mean() + meia})
        print(f"  {m:9s}: {v.mean():.4f} ± {dp:.4f}   IC95% [{v.mean() - meia:.4f}, {v.mean() + meia:.4f}]")
    df_resumo = pd.DataFrame(resumo).set_index('Métrica')

    aderencia = exemplo['aderencia']
    print(f"\n>> CONDICIONAMENTO POR LIMIARES (semente {exemplo['semente']}): τ pedido × D(G_k(z, τ)) obtido")
    for g, grupo in aderencia.groupby('gerador'):
        corr = np.corrcoef(grupo['tau'], grupo['D'])[0, 1] if grupo['D'].std() > 0 else float('nan')
        print(f"  {g}: D médio {grupo['D'].mean():.3f} | erro absoluto médio {np.abs(grupo['D'] - grupo['tau']).mean():.3f} "
              f"| correlação τ×D {corr:+.3f}")

    print(f"\n>> SMPC: {df['SMPC (ms/rodada)'].mean():.0f} ms por rodada | erro máx. vs FedAvg em claro "
          f"{df['Erro máx. SMPC'].max():.1e} | {df_hist['bytes_cliente'].iloc[0] / 1e6:.1f} MB enviados por cliente por roda

    arquivo = f'resultados_teste4_{nome_base}.csv'
    df.to_csv(arquivo)
    print(f"Resultados por semente salvos em '{arquivo}'.")
    resultados_gerais[nome_base] = dict(por_semente=df, resumo=df_resumo, historico=df_hist, aderencia=aderencia)

    # ----- Visualizações -----
    fig, axes = plt.subplots(2, 3, figsize=(22, 11))
    fig.suptitle(f"Teste 4 (v5): STEP-GAN Federado + SMPC ({nome_base}, {len(SEMENTES)} sementes)",
                 fontsize=14, fontweight='bold')

    # 1. Métricas: média ± desvio, com os pontos de cada semente
    ax = axes[0, 0]
    ax.bar(METRICAS, df_resumo['Média'], yerr=df_resumo['Desvio-padrão'], capsize=5, color='#2ca02c', alpha=0.7)
    for i, m in enumerate(METRICAS):
        ax.scatter(np.full(len(df), i) + np.linspace(-0.15, 0.15, len(df)), df[m], color='black', s=15, zorder=3)
    ax.set_ylim(0, 1.05)
    ax.set_title('Métricas no teste (média ± desvio; pontos = sementes)', fontweight='bold')

    # 2. Convergência do FL
    ax = axes[0, 1]
    for semente, grupo in df_hist.groupby('semente'):
        ax.plot(grupo['rodada'], grupo['pr_auc_val'], color='#1f77b4', alpha=0.3)
    media = df_hist.groupby('rodada')['pr_auc_val'].mean()
    ax.plot(media.index, media.values, color='#1f77b4', lw=2.5, marker='o', label='Média entre sementes')
    ax.set_title('PR-AUC na validação por rodada federada', fontweight='bold')
    ax.set_xlabel('Rodada')
    ax.legend()

    # 3. Condicionamento por limiares
    ax = axes[0, 2]
```

```
    cores_g = ['#d62728', '#ff7f0e', '#9467bd']
    for (g, grupo), cor in zip(aderencia.groupby('gerador'), cores_g):
        ax.scatter(grupo['tau'], grupo['D'], s=4, alpha=0.3, color=cor, label=g)
    ax.plot([0, 1], [0, 1], ':', color='gray', label='Ideal: D = τ')
    ax.axhline(ALVO_NORMAL, color='#2ca02c', ls='--', lw=1, label='Alvo normal (0.9)')
    ax.set_xlabel('Limiar pedido τ')
    ax.set_ylabel('Saída do Discriminador D(G_k(z, τ))')
    ax.set_title(f'Condicionamento por limiares (semente {exemplo["semente"]})', fontweight='bold')
    ax.legend(fontsize=8, markerscale=3)

    # 4. Distribuição dos escores
    ax = axes[1, 0]
    s, y = exemplo['av']['scores'], dados['y_test']
    sns.histplot(s[y == 0], color='#1f77b4', bins=50, alpha=0.6, label='Legítima', ax=ax, stat='density')
    sns.histplot(s[y == 1], color='#d62728', bins=50, alpha=0.6, label='Fraude', ax=ax, stat='density')
    ax.axvline(exemplo['av']['limiar'], color='black', ls='--', lw=2, label=f"Limiar: {exemplo['av']['limiar']:.3f}")
    ax.set_yscale('log')
    ax.set_title(f'Escore de anomalia no teste (semente {exemplo["semente"]})', fontweight='bold')
    ax.legend()

    # 5. Matriz de confusão
    ax = axes[1, 1]
    cm = confusion_matrix(y, exemplo['av']['y_pred'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax, cbar=False,
                xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'], annot_kws={"size": 14, "weight": "bold"}
    ax.set_title(f'Matriz de confusão (semente {exemplo["semente"]}, F1 {exemplo["av"]["F1"]:.3f})', fontweight='bold')
    ax.set_ylabel('Real')
    ax.set_xlabel('Previsto')

    # 6. Fidelidade do SMPC
    ax = axes[1, 2]
    for semente, grupo in df_hist.groupby('semente'):
        ax.plot(grupo['rodada'], grupo['erro_max'], marker='o', alpha=0.6, label=f'Semente {semente}')
    ax.axhline(NUM_CLIENTES * 0.5 / SMPC_ESCALA, color='black', ls='--', label='Limite teórico (n · 2^-25)')
    ax.set_yscale('log')
    ax.set_xlabel('Rodada')
    ax.set_title('Erro da agregação SMPC vs FedAvg em claro', fontweight='bold')
    ax.legend(fontsize=8)

    plt.tight_layout()
    plt.show()

print("\n[SUCESSO] Teste 4 (v5) concluído na base BAF!")
```

```
================================================================================
 BASE: BAF | 5 sementes | 5 rodadas x 7 épocas | 3 geradores
================================================================================
Carregando '/root/.cache/kagglehub/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022/versions/2/Base.csv'...
[DADOS] Features: 51 | Treino: 600000 (fraudes: 6617) | Validação: 200000 (fraudes: 2206) | Teste: 200000 (fraudes: 2206)

--- Semente 42 ---
  Rodada 1/5 | PR-AUC val 0.1341 | treino 112.7s | SMPC 215 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.1380 | treino 113.0s | SMPC 250 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.1398 | treino 113.4s | SMPC 221 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.1405 | treino 113.2s | SMPC 219 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.1418 | treino 113.9s | SMPC 227 ms (erro máx. vs claro 8.9e-08)
  -> Rodada escolhida: 5 | ROC-AUC 0.8817 | PR-AUC 0.1333 | F1 0.0187

--- Semente 43 ---
  Rodada 1/5 | PR-AUC val 0.1353 | treino 114.7s | SMPC 221 ms (erro máx. vs claro 8.9e-08)
  Rodada 2/5 | PR-AUC val 0.1381 | treino 114.0s | SMPC 249 ms (erro máx. vs claro 8.9e-08)
  Rodada 3/5 | PR-AUC val 0.1402 | treino 115.5s | SMPC 225 ms (erro máx. vs claro 8.9e-08)
  Rodada 4/5 | PR-AUC val 0.1415 | treino 117.6s | SMPC 245 ms (erro máx. vs claro 8.9e-08)
  Rodada 5/5 | PR-AUC val 0.1424 | treino 116.7s | SMPC 236 ms (erro máx. vs claro 8.9e-08)
```

```
# ======================================================================
# --- 5d. EXPERIMENTO NA BASE INSURANCE CLAIMS (mesma configuração da seção 5) ---
# ======================================================================

METRICAS = ['ROC-AUC', 'PR-AUC', 'Precisão', 'Recall', 'F1', 'Acurácia']
resultados_gerais = globals().get('resultados_gerais', {})  # preserva os resultados das outras bases

for nome_base in DATASETS_INSURANCE:
    print("\n" + "=" * 78)
    print(f" BASE: {nome_base.upper()} | {len(SEMENTES)} sementes | {NUM_RODADAS} rodadas x {EPOCAS_LOCAIS} épocas "
          f"| {len(DEGRAUS)} geradores")
    print("=" * 78)
    dados = carregar_dados(nome_base)
    if dados is None:
        print(f"Aviso: base '{nome_base}' não encontrada em {CAMINHOS[nome_base]}. Pulando.")
        continue

    linhas, historicos, exemplo = [], [], None
    for semente in SEMENTES:
        print(f"\n--- Semente {semente} ---")
        estado_final, rodada_escolhida, historico = executar_federado(dados, semente)
        av = avaliar(estado_final['D'], dados)
        historico['semente'] = semente
        historicos.append(historico)
        linhas.append({'Semente': semente, 'Rodada escolhida': rodada_escolhida, **{m: av[m] for m in METRICAS},
                       'TPS': av['TPS'], 'SMPC (ms/rodada)': 1000 * historico['t_smpc'].mean(),
                       'Erro máx. SMPC': historico['erro_max'].max()})
        print(f"  -> Rodada escolhida: {rodada_escolhida} | ROC-AUC {av['ROC-AUC']:.4f} | PR-AUC {av['PR-AUC']:.4f} "
              f"| F1 {av['F1']:.4f}")
        if exemplo is None:
            exemplo = dict(semente=semente, av=av,
                           aderencia=aderencia_condicionamento(estado_final, dados['input_dim']))

    df = pd.DataFrame(linhas).set_index('Semente')
    df_hist = pd.concat(historicos, ignore_index=True)

    print("\n>> RESULTADOS POR SEMENTE (teste; limiar escolhido na validação):")
    print(df.to_string(formatters={'Erro máx. SMPC': lambda x: f"{x:.1e}", 'TPS': lambda x: f"{x:,.0f}"},
                       float_format=lambda x: f"{x:.4f}"))

    print(f"\n>> MÉDIA ± DESVIO-PADRÃO ({len(SEMENTES)} sementes) e IC 95% (t de Student):")
    resumo = []
    for m in METRICAS:
        v = df[m].values
        dp = v.std(ddof=1) if len(v) > 1 else 0.0
        meia = stats.t.ppf(0.975, len(v) - 1) * dp / np.sqrt(len(v)) if len(v) > 1 else 0.0
        resumo.append({'Métrica': m, 'Média': v.mean(), 'Desvio-padrão': dp,
                       'IC95% inf': v.mean() - meia, 'IC95% sup': v.mean() + meia})
        print(f"  {m:9s}: {v.mean():.4f} ± {dp:.4f}   IC95% [{v.mean() - meia:.4f}, {v.mean() + meia:.4f}]")
    df_resumo = pd.DataFrame(resumo).set_index('Métrica')

    aderencia = exemplo['aderencia']
    print(f"\n>> CONDICIONAMENTO POR LIMIARES (semente {exemplo['semente']}): τ pedido × D(G_k(z, τ)) obtido")
    for g, grupo in aderencia.groupby('gerador'):
        corr = np.corrcoef(grupo['tau'], grupo['D'])[0, 1] if grupo['D'].std() > 0 else float('nan')
        print(f"  {g}: D médio {grupo['D'].mean():.3f} | erro absoluto médio {np.abs(grupo['D'] - grupo['tau']).mean():.3f}
              f"| correlação τ×D {corr:+.3f}")

    print(f"\n>> SMPC: {df['SMPC (ms/rodada)'].mean():.0f} ms por rodada | erro máx. vs FedAvg em claro "
          f"{df['Erro máx. SMPC'].max():.1e} | {df_hist['bytes_cliente'].iloc[0] / 1e6:.1f} MB enviados por cliente por roc

    arquivo = f'resultados_teste4_{nome_base}.csv'
```

```
        df.to_csv(arquivo)
        print(f"Resultados por semente salvos em '{arquivo}'.")
        resultados_gerais[nome_base] = dict(por_semente=df, resumo=df_resumo, historico=df_hist, aderencia=aderencia)

        # ----- Visualizações -----
        fig, axes = plt.subplots(2, 3, figsize=(22, 11))
        fig.suptitle(f"Teste 4 (v5): STEP-GAN Federado + SMPC ({nome_base}, {len(SEMENTES)} sementes)",
                     fontsize=14, fontweight='bold')

        # 1. Métricas: média ± desvio, com os pontos de cada semente
        ax = axes[0, 0]
        ax.bar(METRICAS, df_resumo['Média'], yerr=df_resumo['Desvio-padrão'], capsize=5, color='#2ca02c', alpha=0.7)
        for i, m in enumerate(METRICAS):
            ax.scatter(np.full(len(df), i) + np.linspace(-0.15, 0.15, len(df)), df[m], color='black', s=15, zorder=3)
        ax.set_ylim(0, 1.05)
        ax.set_title('Métricas no teste (média ± desvio; pontos = sementes)', fontweight='bold')

        # 2. Convergência do FL
        ax = axes[0, 1]
        for semente, grupo in df_hist.groupby('semente'):
            ax.plot(grupo['rodada'], grupo['pr_auc_val'], color='#1f77b4', alpha=0.3)
        media = df_hist.groupby('rodada')['pr_auc_val'].mean()
        ax.plot(media.index, media.values, color='#1f77b4', lw=2.5, marker='o', label='Média entre sementes')
        ax.set_title('PR-AUC na validação por rodada federada', fontweight='bold')
        ax.set_xlabel('Rodada')
        ax.legend()

        # 3. Condicionamento por limiares
        ax = axes[0, 2]
        cores_g = ['#d62728', '#ff7f0e', '#9467bd']
        for (g, grupo), cor in zip(aderencia.groupby('gerador'), cores_g):
            ax.scatter(grupo['tau'], grupo['D'], s=4, alpha=0.3, color=cor, label=g)
        ax.plot([0, 1], [0, 1], ':', color='gray', label='Ideal: D = τ')
        ax.axhline(ALVO_NORMAL, color='#2ca02c', ls='--', lw=1, label='Alvo normal (0.9)')
        ax.set_xlabel('Limiar pedido τ')
        ax.set_ylabel('Saída do Discriminador D(G_k(z, τ))')
        ax.set_title(f'Condicionamento por limiares (semente {exemplo["semente"]})', fontweight='bold')
        ax.legend(fontsize=8, markerscale=3)

        # 4. Distribuição dos escores
        ax = axes[1, 0]
        s, y = exemplo['av']['scores'], dados['y_test']
        sns.histplot(s[y == 0], color='#1f77b4', bins=50, alpha=0.6, label='Legítima', ax=ax, stat='density')
        sns.histplot(s[y == 1], color='#d62728', bins=50, alpha=0.6, label='Fraude', ax=ax, stat='density')
        ax.axvline(exemplo['av']['limiar'], color='black', ls='--', lw=2, label=f"Limiar: {exemplo['av']['limiar']:.3f}")
        ax.set_yscale('log')
        ax.set_title(f'Escore de anomalia no teste (semente {exemplo["semente"]})', fontweight='bold')
        ax.legend()

        # 5. Matriz de confusão
        ax = axes[1, 1]
        cm = confusion_matrix(y, exemplo['av']['y_pred'])
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax, cbar=False,
                    xticklabels=['Normal', 'Fraude'], yticklabels=['Normal', 'Fraude'], annot_kws={"size": 14, "weight": "bold'
        ax.set_title(f'Matriz de confusão (semente {exemplo["semente"]}, F1 {exemplo["av"]["F1"]:.3f})', fontweight='bold')
        ax.set_ylabel('Real')
        ax.set_xlabel('Previsto')

        # 6. Fidelidade do SMPC
        ax = axes[1, 2]
        for semente, grupo in df_hist.groupby('semente'):
            ax.plot(grupo['rodada'], grupo['erro_max'], marker='o', alpha=0.6, label=f'Semente {semente}')
        ax.axhline(NUM_CLIENTES * 0.5 / SMPC_ESCALA, color='black', ls='--', label='Limite teórico (n · 2^-25)')
        ax.set_yscale('log')
        ax.set_xlabel('Rodada')
        ax.set_title('Erro da agregação SMPC vs FedAvg em claro', fontweight='bold')
        ax.legend(fontsize=8)

        plt.tight_layout()
        plt.show()

print("\n[SUCESSO] Teste 4 (v5) concluído na base Insurance Claims!")
```